

Algorithms for Data Science

Lecture #1: Introduction

Mateusz Buczyński

University of Warsaw

18.02.2026

Thanks to dr Krzysztof Fleszar for the base material.

Algorithms for Data Science

Goal

“Data Science is not just about libraries. It is about feeding the processor efficiently.”

- From silicon → Python: why hardware matters for algorithms
- Efficiency as a first-class design constraint

Topics

- Lectures 1–3: [Introduction](#)
- Lectures 4–6: [Algorithms](#)
- Lectures 7–8: [Data Structures](#)
- Lecture 9: [Extra](#)

About the instructor

Who I am

- Mateusz Buczyński
- Focus: practical usage of data science in business; time series predictions

Contact

- Email: mp.buczynski2@uw.edu.pl
- Office hours (online): on request (MS Teams; link on Moodle)
- Best way to reach me: email with subject prefix [AFDS]

Logistics & Organization

The schedule

- Moodle link
- Lecture - Wednesdays 15:00, recording available, 15 lectures in total.
- Labs - Wednesdays 16:45 and 18:30, recording available.
- Labs - every other week, 7 labs in total.

Exercise schedule

#	Week type	Date
1	$\ominus$	18.02.2026
2	E	25.02.2026
3	O	04.03.2026
4	E	11.03.2026
5	O	18.03.2026
6	E	25.03.2026
7	O	01.04.2026
7'	E	08.04.2026
8	O	15.04.2026
9	E	22.04.2026
10	O	29.04.2026
11	E	06.05.2026
12	O	13.05.2026
13	E	20.05.2026
14	O	27.05.2026
15	E	03.06.2026

Passing criteria

To pass the labs

- Pass **6 programming assignments (PT)** - 10 points each.
- Pass the **final presentation (FP)** - 60 points.

To pass the whole course

- Pass the labs, and
- Pass the **exam (EX)** - 180 points.

Points split in final grade

- 20% PT
- 20% FP
- 60% EX

You can obtain +1 point to your final grade for activity during the lecture and the labs.

How to learn (resources)

Books

- The “Bible”: *Introduction to Algorithms* (Cormen, Leiserson, Rivest, Stein)
- Visual/intuitive: *Grokking Algorithms* (Aditya Bhargava) — great for beginners

Practice (mental sport)

- Competitive programming:
 - HackerRank (learn syntax)
 - Codeforces (algorithms)
- Tip: treat coding like a sport — build muscle memory

The puzzle: “Guess my number”

The puzzle: “Guess my number”

Scenario

I have a secret number x between 0 and 1000.

The puzzle: “Guess my number”

Scenario

I have a secret number x between 0 and 1000.

Constraint

You can only ask questions of the form: “Is $x \leq \dots$?”

The puzzle: “Guess my number”

Scenario

I have a secret number x between 0 and 1000.

Constraint

You can only ask questions of the form: “Is $x \leq \dots$?”

Strategy

- Don't guess randomly
- Ask “Is $x \leq 500$?” $\rightarrow$ eliminate half instantly
- Repeat

The puzzle: “Guess my number”

Scenario

I have a secret number x between 0 and 1000.

Constraint

You can only ask questions of the form: “Is $x \leq \dots$?”

Strategy

- Don't guess randomly
- Ask “Is $x \leq 500$?” $\rightarrow$ eliminate half instantly
- Repeat

Result: about 10 questions since $2^{10} = 1024$.

Translating logic to code

Variables

l (left bound), r (right bound), m (middle)

Loop

While the range is valid ($l < r$):

- compute m
- if $x \leq m$: answer is in left half $\Rightarrow r = m$
- else: answer is in right half $\Rightarrow l = m + 1$

Crucial logic - loop invariant

Why $l = m + 1$? Otherwise the range might not shrink $\rightarrow$ infinite loop.

Formal definition of an algorithm

Definition

An algorithm is a finite, unambiguous sequence of steps that:

- takes an input from a specified set of valid instances,
- produces the correct output for each valid instance,
- terminates after a finite number of steps.

In this lecture

Binary search is an algorithm for the ordered-search problem.

Application: insertion point

Problem

Given a sorted list a , find the first index i such that $a[i] \geq v$.

Boundary handling

- Use $l = 0$ and $r = \text{len}(a)$
- Handles “all elements $< v$ ” correctly (answer should be n)

Python implementation

```
def binary_search(a, v):  
    l, r = 0, len(a)  
    while l < r:  
        m = (l + r) // 2  
        if a[m] >= v:  
            r = m  
        else:  
            l = m + 1  
    return l
```

Note: in Python, use // for integer division; / produces floats.

How do we know it works? (Invariants)

The problem

Bugs happen at the edges: infinite loops, off-by-one errors.

The tool: loop invariants

A *loop invariant* is a statement that is true:

- before the loop starts,
- after every iteration,
- when the loop ends.

Proving binary search

Invariant

$l \leq \text{result} \leq r$ (the answer is trapped in the current window).

- Initialization: true at start (answer in $0 \dots n$)
- Maintenance: each step shrinks the window ($r - l$ decreases), answer stays inside
- Termination: loop ends at $l = r$; since answer is trapped, l is the answer

The “lie” of computer science

RAM model

We are taught that accessing any memory address takes the same time ($O(1)$).

Reality

This is false for high-performance data science.

New goal

Stop thinking about “steps” and start thinking about **data movement**.

The cost of latency (the hierarchy)

- Registers (brain): < 1 ns
- L1/L2 cache (pocket): $\sim 1\text{--}10$ ns
- RAM (library): ~ 100 ns
- Disk (moon): $\sim 10,000,000$ ns

Takeaway

The CPU often waits for data; performance is the art of minimizing this wait.

Why is `sum(list)` slow? (boxed integers)

C/NumPy integer

- 4 bytes (raw binary)

Python integer

- ~ 28 bytes (PyObject)
- contains: refcount + type + size + value

The cost

Add $\rightarrow$ unwrap two boxes, check types, add, then wrap the result in a new box.

Visual intuition: a warehouse of boxes vs. a stream of raw data.

Memory layout: lists vs. arrays

Python list

- A list of pointers
- Data scattered in RAM (pointer chasing)
- Cache misses $\rightarrow$ CPU waits ~ 100 ns per item

NumPy array

- Contiguous block of memory
- Data lined up $\rightarrow$ cache hits
- CPU can predict the next number

Key concept

Locality of reference.

Vectorization & SIMD

SIMD

Single Instruction, Multiple Data.

- Modern CPUs can add many pairs of numbers per cycle (e.g., 8)
- Python loop: adds 1 pair at a time
- NumPy: uses SIMD to process chunks efficiently

Summary

```
import numpy isn't magic;
```


Linear vs. binary search

Linear search

- Check one-by-one
- Complexity: N steps

Binary search

- Halve the problem
- Complexity: $\log_2 N$ steps

The “technological disadvantage” experiment

The race: processing $N = 1,000,000,000$ items

- Contestant A: supercomputer (10^9 ops/sec) using linear search
 - Contestant B: 1980s BASIC machine (1000 ops/sec) using binary search
-
- A (linear): 10^9 steps $\rightarrow \approx 1$ second
 - B (binary): ≈ 30 steps $\rightarrow 30/1000 = 0.03$ seconds

Conclusion

A smart algorithm on a “dinosaur” beats brute force on a supercomputer.

Summary

- Algorithmic thinking: divide and conquer (binary search)
- Correctness: use invariants to prove your logic
- Performance: respect the hardware (cache & locality)
- Next week: time complexity and Big- O notation

Algorithms for Data Science

Lecture #2: Running Time

Mateusz Buczyński

University of Warsaw

25.02.2026

Thanks to dr Krzysztof Fleszar for the base material.

Recap: What's your problem?!

Definition: Computational Problem

A computational problem is:

- a set of allowed inputs (instances), and
- a correct output required for each allowed input.

Input

- integer array a sorted nondecreasingly, s.t. $a[0] \leq a[1] \leq \dots \leq a[n-1]$
- integer target value v

Output

min i such that $v \leq a[i]$, or n if such index does not exist.

Example: Search Problem

Recap: Algorithm

An algorithm is a finite, unambiguous sequence of steps that:

- takes an input from a specified set of valid instances,
- produces the correct output for each valid instance,
- terminates after a finite number of steps.

Binary search

repeatedly split range in two.

Linear search

scan from left to right.

Compare algorithms?

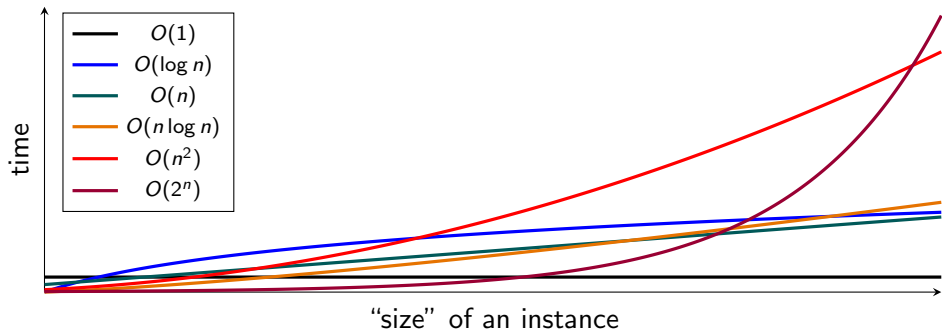
Two main questions

- How fast is the algorithm?
- How much memory does it use?

Focus of this lecture

We focus on running time today. Memory complexity is to be tackled on the next lectures.

Typical Time Complexity Growth



Time Complexity

Core idea

Running time should be analyzed as a function of input size, not as one measured runtime.

Example

For the search problem, natural input size is the array length n .

- Small n : many algorithms appear fast.
- Large n : growth rate dominates.

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best)

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations,
given same probability of each
instance;

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) -
 $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations,
given same probability of each
instance;
- Worst case (guaranteed upper
bound)

Time function

Definition

Time complexity (running time) is a function $f(n)$, where n is input size.

```
# Linear Search
i = 0
while i < n and a[i] < v:
    i += 1
```

- Optimistic case (best) - $a[0] = v \Rightarrow 0$ iterations;
- Average case - $\approx n/2$ iterations, given same probability of each instance;
- Worst case (guaranteed upper bound) $a[n - 1] < v \Rightarrow n$ iterations.

Course convention

Unless stated otherwise, we use **worst-case running time**.

- For a fixed input size n , many instances exist.
- Worst-case picks the maximum runtime among those instances.
- This gives a robust guarantee for planning.

Slow programs

Theory

An algorithm can have poor worst-case bounds.

Practice

The same algorithm can still work very well on many real datasets.

What time measure?

1) Real time?

Use seconds / milliseconds / nanoseconds from wall-clock measurements.

Problem

Wall-clock time depends on hardware, software stack, and other technical details.

- Not universal across machines and years.
- Useful for benchmarking, not enough for theory.

What time measure?

2) Number of high-level operations?

Looks appealing, but operation costs may differ dramatically.

```
tab = [i for i in range(1000000)]  
for i in range(100000):  
    tab.pop(0)
```

```
tab = [i for i in range(1000000)]  
for i in range(100000):  
    tab.pop(len(tab)-1)
```

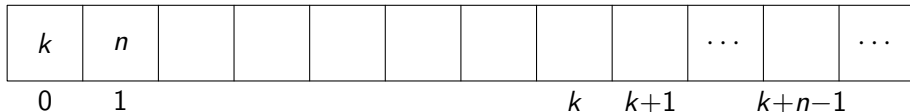
Both remove 100k elements, but runtime differs a lot due to different underlying costs.

Random Access Machine (RAM)

RAM model

A theoretical model where primitive operations have uniform unit cost.

- Hardware-independent abstraction.
- Good for asymptotic reasoning.



k - start position, n - size of array.

Random Access Machine (RAM)

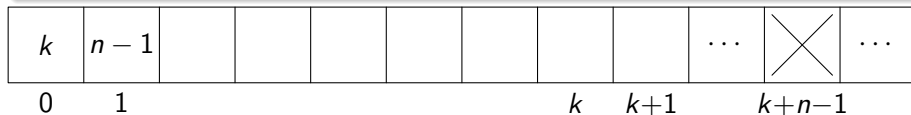
Example 1

Remove last element.

Random Access Machine (RAM)

Example 1

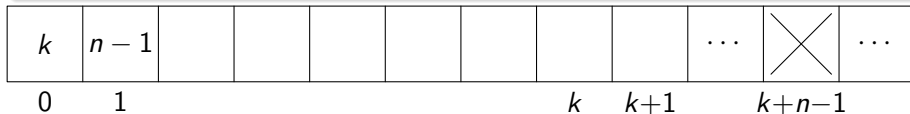
Remove last element.



Random Access Machine (RAM)

Example 1

Remove last element.



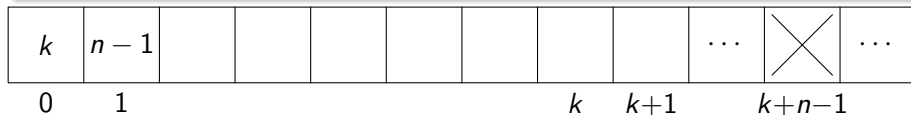
Example 2

Remove first element.

Random Access Machine (RAM)

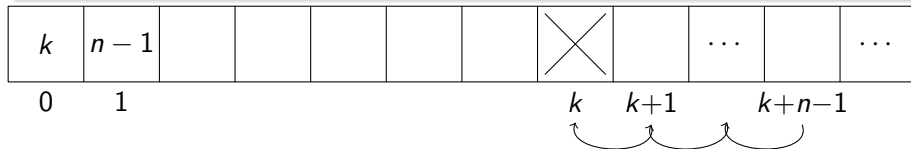
Example 1

Remove last element.



Example 2

Remove first element.



Random Access Machine (RAM)

```
1. R[3] = 0
2. R[4] = R[3] - R[1]
3. R[4] = R[4] + 1
4. IF_LARGER_ZERO R[4] JUMP_TO_LINE 11
5. R[4] = R[0] + R[3]
6. R[4] = R[R[4]] - R[2]
7. R[4] = R[4] + 1
8. IF_LARGER_ZERO R[4] JUMP_TO_LINE 11
9. R[3] = R[3] + 1
10. JUMP_TO_LINE 2
11. RETURN R[3]
```

where $R[0], R[1]$ define $\$a\$$ and $R[2]$ defines $\$v\$$.

Random Access Machine (RAM)

Why not write everything in RAM code?

It is precise, but verbose and hard to read.

- RAM-level listing can look complicated.
- You do not need to memorize this syntax.

The key message: low-level detail exists, but we need a practical analysis layer.

What time measure?

3) Number of RAM operations?

Good: proportional to real time and less tied to specific technology.

Bad: writing full RAM code is cumbersome.

Need a compromise

We want something readable like high-level code but still proportional to runtime.

What time measure?

Goal: proportional to real time

Let $f(n)$ be our analytical cost and $g(n)$ real time.

We accept bounded distortion:

$$\exists c_1, c_2 > 0 : \quad c_1 \leq \frac{g(n)}{f(n)} \leq c_2$$

for sufficiently large n .

Constants may depend on technology and implementation details.

What time measure?

Weighted high-level operations

Assign each high-level operation a weight proportional to RAM effort.

- Easy to write and read
- Simplified operation costs
- Keep proportionality to RAM cost
- Derive complexity from weighted totals

How to know operation costs?

Use lecture notes and language/library references.

What time measure?

4) Total cost of dominating operations

Start with weighted high-level costs, then simplify to a small set of dominating operations.

```
# Binary Search & Destroy
l, r = 0, n
while l < r:
    m = (l + r) // 2
    if a[m] <= v:
        r = m
    else:
        l = m + 1
if l < n:
    a.pop(l)
```

- Simplify per block.
- Then keep dominating ops only.
- Here: $\log_2 n + n$.

What time measure?

Ignore factors: asymptotics

Compare growth rates, not constant multipliers.

A: one loop with many constant-time updates

```
for i in range(0, n):  
    a += 1  
    b += 1  
    ...  
    z += 1
```

B: two loops

```
for i in range(0, n):  
    a += 1  
for i in range(0, n):  
    b += 1
```

n vs $2n$ dominating ops differ by constant factor only, so asymptotically equal.

5. Use Asymptotics Notation

Motivation

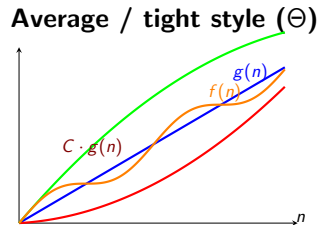
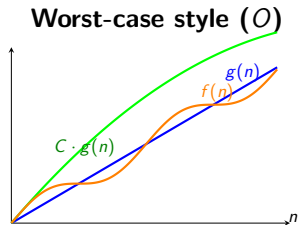
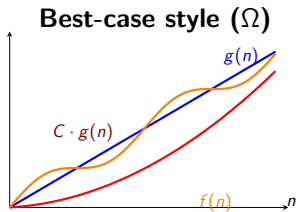
Asymptotics makes analysis and comparison simpler by focusing on dominant growth.

Examples

$$f(n) = 3 + n^2 + 1000n = O(n^2), \quad g(n) = 1000^{1000} + n^2 + \frac{1}{1000}n^3 = O(n^3)$$

- $f(n) = O(g(n))$: upper bound
- $f(n) = \Omega(g(n))$: lower bound
- $f(n) = \Theta(g(n))$: tight bound

Visual Intuition: Best, Worst, Average



Intuition only: worst-case upper bound, best-case lower bound, and tight growth band.

Asymptotic Definitions

- $f(n) = O(g(n))$

If there exist $C > 0$ and n_0 such that

$$f(n) \leq C \cdot g(n) \quad \text{for all } n \geq n_0$$

- $f(n) = \Omega(g(n))$

If $g(n) = O(f(n))$.

- $f(n) = \Theta(g(n))$

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

$$f(n) = n^2$$

Typical Complexities

Constant complexity

$$f(n) = O(1) \iff f(n) \leq \text{some constant}$$

- Runtime does not grow with input size.
- Constant factors still exist, but growth class stays constant.

$$f(n) = 1000$$

$$f(n) = n^2$$

$$f(n) = \frac{1}{n}$$

Typical Complexities

Polynomial complexity

$$f(n) = O(n^d) \text{ for constant } d > 0$$

Equivalent shorthand: $f(n) = n^{O(1)}$.

Dominant term rule

$$n^7 + 4n^8 + 10n^{99} = \Theta(n^{99})$$

Compare exponentials

$$n^2 = O(n^{2.5}), \quad n^2 \neq \Omega(n^{2.5}), \quad n^2 \neq \Theta(n^{2.5})$$

Typical Complexities

Exponential complexity

$$f(n) = O(d^n) \text{ for constant } d > 1$$

Compare exponentials

$$3^n \neq O(2^n), \quad 3^n = \Omega(2^n), \quad 3^n \neq \Theta(2^n)$$

Typical Complexities

More examples

$$5 = O(n^2), \quad n^2 = O(n^{10}), \quad n^{10} = O(2^n), \quad 2^n = O(3^n)$$

Hence by transitivity: $5 = O(3^n)$.

Rule (transitivity of O)

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Definitions recap: O (upper), Ω (lower), Θ (tight).

Memory complexity

- Number of memory cells
- Asymptotic notation defined analogously to running time

Examples? Next lecture!

Algorithms for Data Science

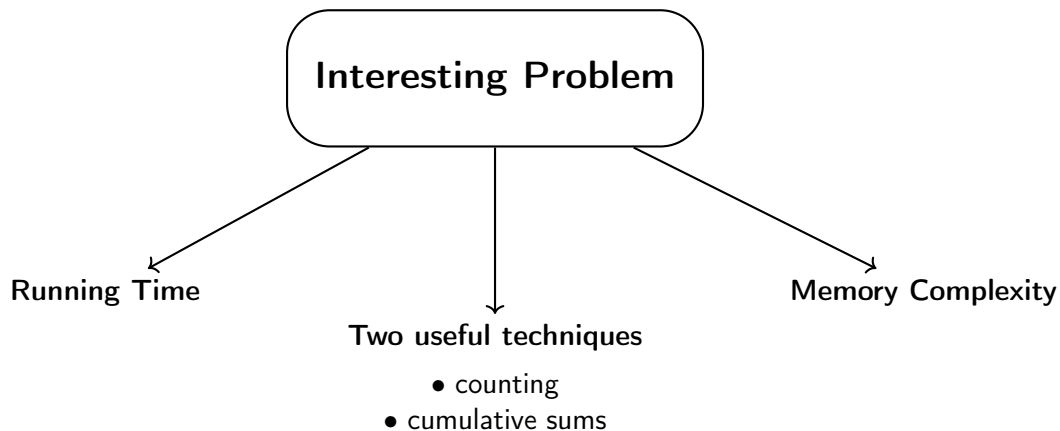
Lecture #3 #4: Analyzing Algorithms

Mateusz Buczyński

University of Warsaw

04, 11.03.2026

Thanks to dr Krzysztof Fleszar for the base material.



Recap: Running Time vs Memory Complexity

Running Time

- Number of **basic operations**.
- Examples: $+$, $-$, $/$, $\times$, $=$, $\leq$, $\dots$

Memory Complexity

- Number of **memory cells** used (RAM model).
- Intuition: how many values we store **at the same time**.

Common perspective

Both are studied as functions of the input size n (or another key parameter).

Recap: Example

Example

Create an array of size n filled with 0s.

Reasoning

- Allocate n cells.
- Write 0 into each cell once.

Recap: Example

Example

Create an array of size n filled with 0s.

Reasoning

- Allocate n cells.
- Write 0 into each cell once.

Complexities

- Time: $\Theta(n)$
- Space: $\Theta(n)$

“Oh”-Notation: Picking Useful Bounds

For a running-time function $f(n)$

- $f(n) = O(g(n))$: g is an asymptotic upper bound
- $f(n) = \Omega(g(n))$: g is an asymptotic lower bound
- $f(n) = \Theta(g(n))$: asymptotically tight (both upper and lower)

Example: $f(n) = 10n + n^2$

$$10n + n^2 = \Omega(1), \quad 10n + n^2 = O(1000n^{1000})$$

These are correct, but not very helpful.

Our goal

Find bounds that match as closely as possible:

$\Omega(\cdot)$ and $O(\cdot)$ meet at $\Theta(\cdot)$.

"Oh"-notation - example

Sandwich the function

Let $f(n) = n^2 + 10n$.

For sufficiently large n :

$$n^2 \leq 10n + n^2 \leq 10n^2 + n^2 = 11n^2$$

$$10n + n^2 = \Omega(n^2)$$

$$10n + n^2 = O(n^2)$$

$$\Rightarrow 10n + n^2 = \Theta(n^2)$$

Interpretation

$O(\cdot)$ is commonly used for worst-case upper bounds on running time, $\Omega(\cdot)$ gives lower bounds, and $\Theta(\cdot)$ gives the tight growth rate.

Asymptotic Notation Is Not Symmetric

Unlike ordinary equality

Ordinary equality is bidirectional:

$$f(n) = g(n) \iff g(n) = f(n).$$

But asymptotic statements are directional:

$$f(n) = O(g(n)) \implies g(n) = O(f(n)).$$

Counterexample

$$n = O(n^2) \quad \text{but} \quad n^2 \neq O(n).$$

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$\text{tmp} = k + m$

$\text{tmp} = \text{tmp}/2$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$\text{tmp} = k + m$

$\text{tmp} = \text{tmp}/2$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

Time = $\Theta(1)$,

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$\text{tmp} = k + m$

$\text{tmp} = \text{tmp}/2$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

Time = $\Theta(1)$, Extra Space = $\Theta(1)$

Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
```

```
for i = 1..n/2:
```

```
    r += a[i]
```

```
return r
```

Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
for i = 1..n/2:
    r += a[i]
return r
```

Cost

- About $n/2$ loop updates $\Rightarrow \Theta(n)$ time.
- Extra variables: r and $i \Rightarrow \Theta(1)$ space.

B) Prefix-sum array

Input: $a[1..n]$

```
b[1..n] initialized with 0
for i = 1..n:
    b[i] = a[i]
for i = 2..n:
    b[i] += b[i-1]
return b[n]
```


Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
for i = 1..n/2:
    r += a[i]
return r
```

Cost

- About $n/2$ loop updates $\Rightarrow \Theta(n)$ time.
- Extra variables: r and $i \Rightarrow \Theta(1)$ space.

B) Prefix-sum array

Input: $a[1..n]$

```
b[1..n] initialized with 0
for i = 1..n:
    b[i] = a[i]
for i = 2..n:
    b[i] += b[i-1]
return b[n]
```

Cost

- Dominating operations: $n + (n - 1)$ (plus initialization) $\Rightarrow \Theta(n)$ time.
- Stores array $b[1..n]$ (and loop index) $\Rightarrow \Theta(n)$ space.

More Parameters

Input with two independent sizes

$$a[0..k-1], \quad b[0..m-1], \quad n = k + m$$

Key idea

When an algorithm depends on both k and m , keeping both parameters can give a more informative comparison than using only n .

Example: Two Similar Worst Cases, Different Behavior

Algorithm A

```
[i for i in range(m*m + k)]
```

$$T_A(k, m) = \Theta(m^2 + k)$$

Algorithm B

```
[i for i in range(m + k*k)]
```

$$T_B(k, m) = \Theta(m + k^2)$$

Worst-case in terms of n

- T_A : if $k = 0$, $m = n$, then $T_A = \Theta(n^2)$.
- T_B : if $m = 0$, $k = n$, then $T_B = \Theta(n^2)$.

Teaching takeaway

Both are $\Theta(n^2)$ in the worst case, but $\Theta(m^2 + k)$ vs. $\Theta(m + k^2)$ enables a better comparison for specific k, m regimes.

Main Example Problem: Eye Tracking

Setup

We model a screen (width W , height H) with:

- n gaze points (where users looked),
- k axis-aligned rectangles (regions of interest, e.g., UI elements or ads).

Query / Output

For each rectangle, report how many gaze points fall inside it.

Given k rectangles, compute k counts.

Interpretation

Each rectangle count estimates user attention on that region.

Why This Problem Matters

Applications

- Website design
- User interfaces
- Cockpit design

Applications

- Ad placement
- Gaming

Algorithmic View

This is a geometric counting problem:

points + rectangles $\rightarrow$ count points per rectangle.

Simplified Problem: From 2D to 1D

2D version

- Points in the plane
- Axis-aligned rectangles
- Query: how many points are inside each rectangle?

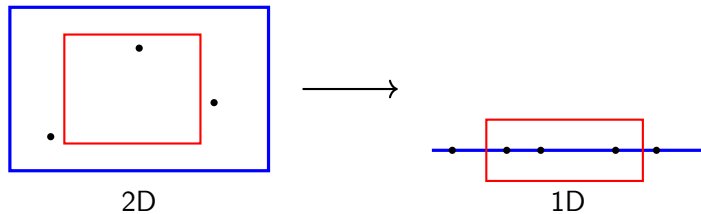
1D version

- Points on a line
- Intervals
- Query: how many points are inside each interval?

Key idea

We simplify geometry: **rectangles** become **intervals**, while the counting goal stays the same.

Simplified Problem



Simplified Problem: Why This Helps

Decision rule

First solve the 1D counting problem cleanly, then reuse the same analysis pattern for 2D.

What is preserved?

- Same type of output: one count per query object
- Same algorithmic question: fast repeated range counting
- Same evaluation criteria: running time and memory

Simplified Problem

Input

- n points on a line segment in $[0, W)$
- k intervals, each with integer endpoints

Output

Compute an array $\text{res}[1..k]$ such that

$\text{res}[i] = \text{number of points in the } i\text{-th interval.}$

Notation reminder

If I denotes the i -th interval, then $\text{res}[I]$ is just shorthand for $\text{res}[i]$.

Example: $n = 8$, $k = 2$

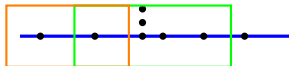
Intervals

- I_1 : first (left) interval
- I_2 : second (right) interval

Counts

$\text{res}[1] = 2$

$\text{res}[2] = 6$



Simplified Problem: Baseline Idea

Input

- n points on a line
- k intervals
- Integer coordinates in $[0, W)$

Output

For each interval I :

$$\text{res}[I] = \#\{\text{points inside } I\}.$$

Solution 1 (naïve)

```
res[1..k] = 0
for each interval I:
  for each point p:
    if p in I:
      res[I]++
```

- k intervals $\times$ n points
- Membership test in $\Theta(1)$

Time: $\Theta(nk)$

Extra space: $\Theta(k)$

Simplified Problem: Counting Approach

Key observation

Coordinates are integers in $[0, W)$, so we can count points per coordinate.

Step 1: Build counts

```
counter[0..W-1] = 0
for each point p:
    x = coordinate of p
    counter[x]++
```

Time: $\Theta(n)$, Space: $\Theta(W)$

Step 2: Answer intervals

```
res[1..k] = 0
for each interval I:
    for each x in I:
        res[I] += counter[x]
```

Time: $\Theta(kW)$ (worst case), Space: $\Theta(k)$

$$\Theta(W + n + k + kW) = \Theta(n + kW)$$

Result

Total Time: $\Theta(n + kW)$

Simplified Problem: Cumulative Sums

Setup

We have:

- n points on integer coordinates $0, 1, \dots, W$
- k interval queries

Let $c[x]$ be the number of points at coordinate x .

Prefix idea

Define

$$P[x] = \sum_{i=0}^{x-1} c[i]$$

So $P[x]$ is the number of points strictly to the left of coordinate x .

Why this helps

Once P is built, each query is answered by one subtraction.

Answering an Interval Query

Decision rule

For interval between coordinates ℓ and r :

$$\#(\text{points in } [\ell, r)) = P[r] - P[\ell]$$

If you want closed intervals

$$\#(\text{points in } [\ell, r]) = P[r + 1] - P[\ell]$$

(assuming integer coordinates).

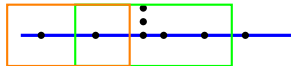
From the picture

If

$$P[\ell] = 1, \quad P[r] = 7$$

then

$$\# \text{ in between} = 7 - 1 = 6.$$



Cumulative Sums Algorithm

Step 1: Count points per coordinate

$\text{counter}[0..W - 1] \leftarrow 0$

For each point p at coordinate x_p :

$\text{counter}[x_p] \leftarrow \text{counter}[x_p] + 1$

Step 2: Build prefix sums

$\text{cumsum}[0] \leftarrow 0$

For $x = 1, \dots, W$:

$\text{cumsum}[x] = \text{cumsum}[x - 1] + \text{counter}[x - 1]$

So $\text{cumsum}[x] = \text{number of points strictly left of } x$.

Complexity

$$\Theta(n) + \Theta(W) + \Theta(k) = \Theta(n + k + W)$$

Space

$\Theta(W)$ for counter, cumsum

Comparison

- Sol 1: $\Theta(n \cdot k)$
- Sol 2: $\Theta(n + k \cdot W)$
- Sol 3: $\Theta(n + k + W)$

Simplified Problem: Formal Statement

Input

Given three integer arrays:

$$x \in \{0, \dots, W - 1\}^n, \quad x_0, x_1 \in \{0, \dots, W - 1\}^k$$

Equivalently, for every value v appearing in x, x_0, x_1 :

$$0 \leq v < W.$$

Output

Compute an integer array res of size k such that

$$\text{res}[i] = |\{j \mid x_0[i] \leq x[j] \leq x_1[i]\}| \quad \text{for } i = 0, \dots, k - 1.$$

What Does $\text{res}[i]$ Count?

i -th query interval

$$[x_0[i], x_1[i]]$$

- The pair $(x_0[i], x_1[i])$ defines the i -th interval.
- $\text{res}[i]$ is the number of positions j in array x whose value lies in that interval.
- Bounds are **inclusive**: $x_0[i] \leq x[j] \leq x_1[i]$.

Simplified Problem: Python

Compact implementation

```
counts = [0 for _ in range(W)]
for i in range(n):
    counts[x[i]] += 1

cumsum = [0 for _ in range(W + 1)]
for p in range(1, W + 1):
    cumsum[p] = cumsum[p - 1] + counts[p - 1]

res = [0 for _ in range(k)]
for j in range(k):
    res[j] = cumsum[x1[j] + 1] - cumsum[x0[j]]
```

Eyetracking Problem: Recall Main 2D Problem

Input

- Screen size: $W \times H$
- n gaze points in the plane
- k axis-aligned query rectangles

Output

For each rectangle i , report:

$$\text{res}[i] = \#\{\text{points inside rectangle } i\}.$$

- Same dataset of points, many rectangle queries.
- Goal: fast counting per rectangle.

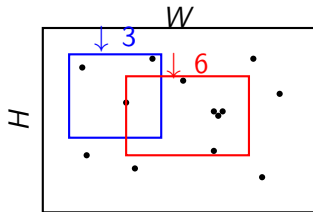
Visual Intuition: Count Points in Each Rectangle

Example query answers

- Blue rectangle contains 3 points.
- Red rectangle contains 6 points.

What changes across queries?

- Points are fixed.
- Rectangle boundaries change.
- We repeat counting k times.



Solution 1: Brute-Force Counting (2D)

Input

- Points: arrays x, y of size n , where point j is $(x[j], y[j])$.
- Rectangles: arrays x_0, y_0, x_1, y_1 of size k , where rectangle i has opposite corners

$$(x_0[i], y_0[i]) \quad \text{and} \quad (x_1[i], y_1[i]).$$

Output

Compute $\text{res}[0..k-1]$, where

$$\text{res}[i] = |\{j : x_0[i] \leq x[j] \leq x_1[i] \wedge y_0[i] \leq y[j] \leq y_1[i]\}|.$$

- Borders are included ($\leq$, not $<$).
- Same points are reused for all k rectangle queries.

Solution 1: Direct Double Loop

Reference implementation (C++ style)

```
vector<int> res(k, 0);

for (int i = 0; i < k; i++) {
    for (int j = 0; j < n; j++) {
        if (x0[i] <= x[j] && x[j] <= x1[i] &&
            y0[i] <= y[j] && y[j] <= y1[i]) {
            res[i]++;
        }
    }
}
```

Time

$$\Theta(k \cdot n)$$

Memory

$\Theta(k)$ for output res

($\Theta(1)$ extra beyond output).

Analyzing Solution 1: Running Time

What the algorithm does

- Create `res` with k entries initialized to 0.
- For each rectangle index $i = 0, \dots, k - 1$, scan all points $j = 0, \dots, n - 1$.
- For each pair (i, j) , check:

$$x_0[i] \leq x[j] \leq x_1[i] \quad \wedge \quad y_0[i] \leq y[j] \leq y_1[i]$$

- If true, increment `res[i]`.

Operation count

Each inner check is constant time: $\Theta(1)$.

Total checks: $k \cdot n$.

$$T(n, k) = k \cdot n \cdot \Theta(1) = \Theta(nk)$$

Analyzing Solution 1: Memory Complexity

Core idea

- The algorithm allocates an output array `res` of size k : `vector<int> res(k, 0)`.
- Two loop counters (i, j) are the only temporary variables.
- Input arrays are provided to the algorithm (not newly allocated inside the loops).

Space contributions

$$S_{\text{input}} = (n, k, w, h) + (x, y) + (x_0, x_1, y_0, y_1) = 4 + 2n + 4k = \Theta(n + k)$$

$$S_{\text{aux}} = \Theta(1) \quad S_{\text{output}} = \Theta(k)$$

What Should We Report?

Auxiliary space

$$S_{\text{aux}} = \Theta(1)$$

- Loop indices i, j
- No other growing temporary structure

Output space

$$S_{\text{output}} = \Theta(k)$$

- Array `res` stores one answer per rectangle

Final complexity statements

- If reporting *extra space including output*: $\Theta(k)$
- If reporting *auxiliary space only*: $\Theta(1)$
- If including input storage: $\Theta(n + k)$

Solution 2: Counting Approach

Initialization

```
vector<vector<int>> counts(H, vector<int>(W, 0));
```

How to read this line

- `counts` is a vector of size H (so: H rows).
- Each row is an `int` vector of size W (so: W columns).
- Every entry starts as 0.

Solution 2: Counting Table Intuition

Structure

$$\text{counts} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix} \quad \begin{array}{l} H \text{ rows} \\ W \text{ columns} \end{array}$$

Meaning of one cell

`counts[b][a]`

stores the number of points located at pixel (a, b) .

Key idea

Count once into `counts`, then answer rectangle queries from table sums.

Counting approach: answer each rectangle query

For rectangle i with corners $(x_0[i], y_0[i])$ to $(x_1[i], y_1[i])$

$\text{sum} \leftarrow 0$

for $b = y_0[i]$ to $y_1[i]$: for $a = x_0[i]$ to $x_1[i]$: $\text{sum} += \text{counts}[b][a]$

Then append the answer for this rectangle:

$\text{res.push_back}(\text{sum})$

What this is doing

For each query rectangle, scan all coordinates inside it and add the number of points stored in each cell.

Solution 2: Counting Approach

Pseudo-code

```
vector<vector<int>> counts(H, vector<int>(W, 0));
for (int i = 0; i < n; i++) counts[y[i]][x[i]]++;

vector<int> res;
for (int i = 0; i < k; i++) {
    int sum = 0;
    for (int b = y0[i]; b <= y1[i]; b++)
        for (int a = x0[i]; a <= x1[i]; a++)
            sum += counts[b][a];
    res.push_back(sum);
}
```

- 'counts[b][a]' = number of points at coordinate (a, b) .
- 'res[i]' = number of points inside rectangle i .

Solution 2: Running Time

Preprocessing

- Build 'counts' array: $\Theta(HW)$
- Insert n points into 'counts': $\Theta(n)$

Per rectangle

- Initialize 'sum': $\Theta(1)$
- Scan all cells in rectangle:

$$\Theta((x_1 - x_0 + 1)(y_1 - y_0 + 1))$$

Worst case

A rectangle can cover almost the full grid:

$$\text{width} = \Theta(W), \quad \text{height} = \Theta(H)$$

So one query costs $\Theta(WH)$

For k rectangles

$$\Theta(kWH)$$

Total

$$\Theta(n + kWH)$$

Two Baseline Approaches

Solution 1: Direct counting

For each rectangle, scan all points and count those inside.

$$\Theta(n \cdot k)$$

Solution 2: Grid counting approach

- Place points into a $W \times H$ counting grid
- For each rectangle, sum covered grid cells

$$\Theta(n + k \cdot W \cdot H)$$

Takeaway

Both are correct baselines; the second shifts work to the grid and can help when many queries are asked on the same screen.

Solution 3: Cumulative Sums

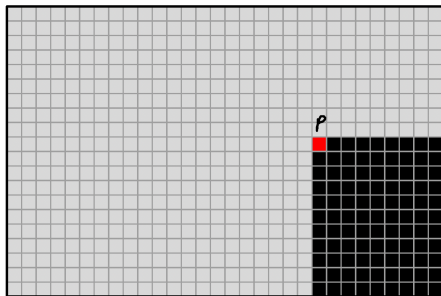
Key idea

Precompute a table so each cell stores the number of points in a large suffix region:

$$S[x, y] = \#\{\text{points in } [x, W) \times [y, H)\}.$$

How to read the picture

- Pick one cell (the red cell).
- Count all points in the black rectangle starting at that cell and extending to the bottom-right corner.
- Save that count as $S[x, y]$.



Solution 3: Cumulative Sums

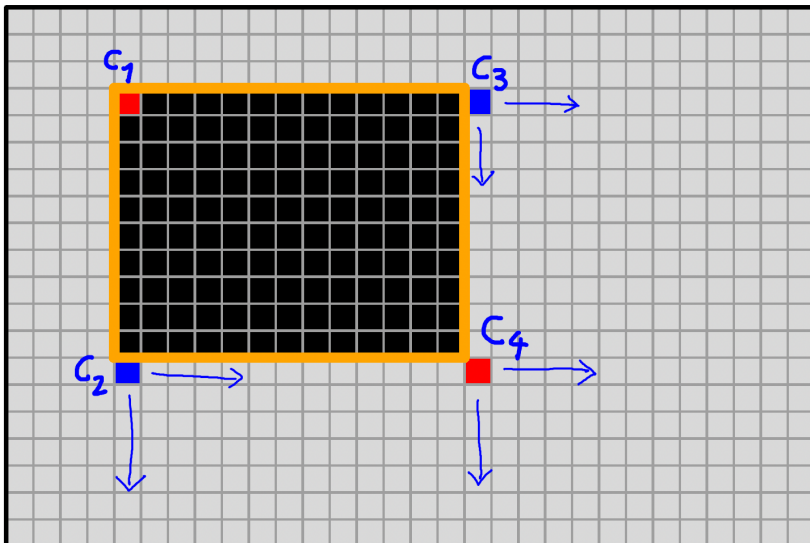
Inclusion–exclusion with four corners

Let c_1 (top-left), c_2 (bottom-left), c_3 (top-right), and c_4 (bottom-right) be the aligned reference corners. Then

$$\#(\text{points in rectangle}) = \text{cumsum}[c_1] - \text{cumsum}[c_2] - \text{cumsum}[c_3] + \text{cumsum}[c_4].$$

- $\text{cumsum}[c_1]$ starts from a larger suffix region that contains the target rectangle.
- Subtracting $\text{cumsum}[c_2]$ and $\text{cumsum}[c_3]$ removes the outside bottom and right parts.
- Their intersection (bottom-right overlap) is subtracted twice, so add back $\text{cumsum}[c_4]$ once.

Solution 3: Cumulative Sums



Solution 3: Query Cost

Per rectangle

The formula uses a constant number of table lookups and arithmetic operations.

$$T_{\text{one query}} = \Theta(1)$$

For k rectangles

$$T_{\text{all queries}} = \Theta(k)$$

Important caveat

This query cost holds **assuming** `cumsum` is already precomputed!.

Solution 3: Cumulative Sums

How do we compute cumulative sums?

We first build a grid that tells us how many points fall on each pixel.

Step 1: Count points on each pixel

For each pixel (a, b) , define

$$\text{counts}[b][a] = \#\{\text{points with coordinates } (a, b)\}.$$

Interpretation

- Most pixels have value 0.
- A pixel with one point has value 1.
- If multiple points share a pixel, the value is greater than 1.

Solution 3: Cumulative Sums

Two arrays to build

- **Step 1:** Compute the number of points *on each pixel*:

$$P(x, y) = \#\{\text{points at pixel } (x, y)\}$$

- **Step 2:** Compute the number of points *to the right of each pixel*:

$$R(x, y) = \#\{\text{points strictly right of } (x, y)\}$$

Solution 3: Cumulative Sums

Three counting steps

- 1 Count points on each pixel (x, y) .
- 2 For each pixel, count points to the right on the same row.
- 3 For each pixel, count points in the full “black area” (suffix rectangle).

Cost of cumsum table

$$\Theta(n + W \cdot H)$$

- Point insertion: $\Theta(n)$
- DP over grid: $\Theta(W \cdot H)$

Key observation

Each pixel update is $\Theta(1)$, and there are $W \cdot H$ pixels.

Solution 3: Final Running Time

Query phase

After preprocessing, each rectangle answer uses a constant number of table lookups:

$$\Theta(1) \text{ per query} \Rightarrow \Theta(k) \text{ for all queries.}$$

Total complexity of Solution 3

$$\Theta(n + W \cdot H) + \Theta(k) = \Theta(n + k + W \cdot H).$$

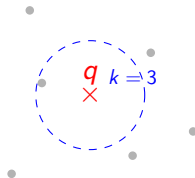
Comparison

$$\text{Sol 1: } \Theta(nk), \quad \text{Sol 2: } \Theta(n + kW \cdot H), \quad \text{Sol 3: } \Theta(n + k + W \cdot H).$$

kNN Search Problem

Input

- Dataset $X = \{x_1, \dots, x_n\}$ in d dimensions
- Query point $q \in \mathbb{R}^d$
- Parameter k (number of neighbors)



Output

The k points from X with the smallest distance to q .

$$\text{dist}(q, x_i) = \|q - x_i\|_2$$

Algorithmic Goal

How do we find these neighbors efficiently as n and d grow?

Stage 1: Brute Force kNN

Algorithm

```
best = [] # max-heap of size k
for i in range(n):
    d_i = compute_dist(q, X[i]) #  $O(d)$ 
    update_heap(best, d_i, i)    #  $O(\log k)$ 
return best
```

Complexity

- Time: $\Theta(n(d + \log k))$
- Extra Space: $\Theta(k)$

Performance

Works for any d , but is **linear in n** .
Too slow for massive datasets.

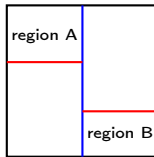
Stage 2: Exact Search (KD-Trees)

Preprocessing: $\Theta(n \log n)$

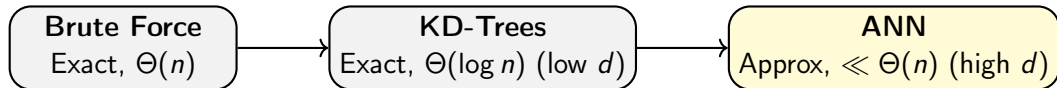
Recursively split space into regions using axis-aligned medians.

Query: $\approx \Theta(\log n + k)$

- Use the tree to prune branches that are too far.
- **High d issue:** "Curse of dimensionality" makes pruning fail ($\Theta(n)$).



Stage 3: Approximate kNN (ANN)



The Trade-off

- We sacrifice **Recall** (finding the 100% true neighbors).
- We gain **Speed** (sublinear query time).

Techniques

- **LSH**: Random projections.
- **Graphs (HNSW)**: Navigating small-world networks.

Takeaway

In Data Science, we almost always use **Stage 3** for production scale.

A może tego lata zrobisz coś
dla milionów użytkowników?
Aplikuj na staż!

Aplikuj do 29 marca!



Ścieżki: Data Science, UX, Project Management, Quality,
AI Fullstack, Backend, Mobile oraz Fullstack Internship.

Algorithms for Data Science

Lecture #5: Divide & Conquer

Mateusz Buczyński

University of Warsaw

18.03.2026

Thanks to dr Krzysztof Fleszar for the base material.

Divide and Conquer

Core idea

Divide a big problem into smaller subproblems.

Typical workflow

- Split the original task into manageable parts.
- Solve each part (often recursively).
- Combine partial solutions into the final answer.

Decision rule

If a problem is too hard to solve directly,
divide it first.

Divide and Conquer: Generic Algorithm

Idea

For a problem of size n , choose between a **base case** and a **recursive case**.

If n is very small

- Solve directly.
- Use a simple method (e.g., brute force).

Otherwise

- Divide into a subproblems.
- Each subproblem has size at most $\frac{n}{b}$.
- Solve subproblems recursively.
- Combine their solutions.

Divide and Conquer: Structure of Work

Recursive pattern

Problem size $n \longrightarrow a$ subproblems of size $\leq \frac{n}{b} \longrightarrow$ combine

Interpretation of parameters

- a : how many smaller problems we create,
- b : how much smaller each one is.

Decision rule

Keep applying the recursive step until the problem becomes small enough to solve directly.

Recall any examples?

Warm-up question

Can you recall examples of divide-and-conquer from:

From the lecture

- Which examples did we discuss in class?
- What was the *divide* step?
- What was the *combine* step?

From exercises

- Which problems used the same pattern?
- What subproblems were solved recursively?
- How was the final answer assembled?

Small Example: Binary Search

Current search interval

We focus on a sorted array segment indexed from l to r .

$$n = r - l + 1$$

Choose the middle index m .

Left half

Indices from l to $m - 1$

Size at most $\frac{n}{2}$

Right half

Indices from $m + 1$ to r

Size at most $\frac{n}{2}$

Key point

At each step, binary search keeps only one of these halves.

Binary Search as Divide-and-Conquer

Recursive decision

If $l < r$ (problem not small), reduce to one subproblem of size $\leq \frac{n}{2}$.

Match to the template

$$a = 1, \quad b = 2$$

- One recursive subproblem is created.
- Its size is at most half of the current interval.

Base case

When the interval is small enough (or empty), stop recursion and return the answer.

Small Example 2: Coins Problem

Setup

Given n coins, split them into three groups so that each group has size

$$\leq \frac{n}{3}.$$

When is the problem not small?

If $n > 1$, we treat it as a non-small problem.

Divide-and-conquer parameters

$$a = 1, \quad b = 3.$$

Reduction step

Reduce to **one** subproblem of size

$$\leq \frac{n}{3}.$$

Big Example: Multiplication

Goal

Compute $12 \cdot 34$ by splitting each number into tens and ones.

$$12 = 1 \cdot 10 + 2, \quad 34 = 3 \cdot 10 + 4$$

$$12 \cdot 34 = (1 \cdot 10 + 2)(3 \cdot 10 + 4)$$

Big Example: Multiplication

$$(1 \cdot 10 + 2)(3 \cdot 10 + 4) = (1 \cdot 3) \cdot 100 + (1 \cdot 4 + 2 \cdot 3) \cdot 10 + (2 \cdot 4)$$

Compute each part

$$(1 \cdot 3) \cdot 100 = 300, \quad (1 \cdot 4 + 2 \cdot 3) \cdot 10 = (4 + 6) \cdot 10 = 100, \quad 2 \cdot 4 = 8$$

$$12 \cdot 34 = 300 + 100 + 8 = 408$$

Takeaway

Split into high/low digits, multiply smaller parts, then combine by powers of 10.

Big Example: Multiplication of Polynomials

Setup

Treat each number as a degree-1 polynomial in x :

$$(1x^1 + 2x^0)(3x^1 + 4x^0).$$

Distribute term by term

$$(1x^1)(3x^1) + (1x^1)(4x^0) + (2x^0)(3x^1) + (2x^0)(4x^0).$$

Goal

Group terms with the same power of x .

Big Example: Multiplication of Polynomials

After grouping equal powers

$$(1x^1 + 2x^0)(3x^1 + 4x^0) = (1 \cdot 3)x^2 + (1 \cdot 4 + 2 \cdot 3)x^1 + (2 \cdot 4)x^0.$$

Coefficient view

- x^2 : $1 \cdot 3$
- x^1 : $1 \cdot 4 + 2 \cdot 3$
- x^0 : $2 \cdot 4$

Takeaway

Polynomial multiplication = multiply all pairs, then sum coefficients of equal powers.

Big Example: Multiplication of Polynomials

Input and output

Given coefficient arrays

$$a[0..n-1], \quad b[0..n-1],$$

compute

$$c[0..2n-2].$$

Definition of each output coefficient

For every $k \in \{0, \dots, 2n-2\}$,

$$c[k] = \sum_{\substack{i,j \\ i+j=k}} a[i] \cdot b[j].$$

Interpretation

To get $c[k]$, add all pairwise products whose indices sum to k .

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

$$c[2] = a[0] \cdot b[2] + a[1] \cdot b[1] + a[2] \cdot b[0]$$

Big Example: Multiplication of Polynomials

First coefficients

$$c[0] = a[0] \cdot b[0]$$

$$c[1] = a[0] \cdot b[1] + a[1] \cdot b[0]$$

$$c[2] = a[0] \cdot b[2] + a[1] \cdot b[1] + a[2] \cdot b[0]$$

Pattern

- Index k defines a diagonal.
- Collect all (i, j) with $i + j = k$.
- Sum their products.

Takeaway

Polynomial multiplication is coefficient convolution.

Warm-Up: Polynomials as Algebra

Addition

$$\begin{aligned}(2x^2 + x) + (3x + 5) \\ = 2x^2 + (x + 3x) + 5 = 2x^2 + 4x + 5\end{aligned}$$

Reminder

$$1 = x^0, \quad 5 = 5x^0$$

Multiplication (Distribute)

$$\begin{aligned}(2x^2 + x)(3x + 5) \\ = 2x^2 \cdot 3x + 2x^2 \cdot 5 + x \cdot 3x + x \cdot 5 \\ = 6x^3 + 10x^2 + 3x^2 + 5x = 6x^3 + 13x^2 + 5x\end{aligned}$$

Coefficient View (for Divide-and-Conquer)

Addition = Component-wise Sum

Use coefficient order $(x^0, x^1, x^2, \dots)$:

$$2x^2 + x \longleftrightarrow [0, 1, 2]$$

$$3x + 5 \longleftrightarrow [5, 3, 0]$$

$$[0, 1, 2] + [5, 3, 0] = [5, 4, 2]$$

$$\Longleftrightarrow 2x^2 + 4x + 5$$

Multiplication = Convolution

$$C(x) = A(x) \cdot B(x)$$

Example:

$$A = [1, 1, 1], \quad B = [1, 2, 2]$$

$$C = [1, 3, 5, 4, 2, 0]$$

(Last zero is just padding.)

Key idea

Polynomial multiplication is not component-wise product; it mixes terms by degree.

Trivial Algorithm: Multiplication of Polynomials

Problem

Given coefficient arrays $a[0..n-1]$ and $b[0..n-1]$, compute $c[0..2n-2]$ such that

$$c[k] = \sum_{\substack{i,j \\ i+j=k}} a[i] \cdot b[j].$$

Direct algorithm

1. Initialize $c[0..2n-2] \leftarrow 0$
2. For $i \leftarrow 0$ to $n-1$ do
3. For $j \leftarrow 0$ to $n-1$ do
4. $c[i+j] \leftarrow c[i+j] + a[i] \cdot b[j]$
5. End For
6. End For

Interpretation

Multiply every pair of coefficients and accumulate by degree index $i+j$.

Runtime of the Trivial Algorithm

Operation count

- Outer loop runs n times.
- Inner loop runs n times per outer iteration.
- Each update

$$c[i + j] \leftarrow c[i + j] + a[i] \cdot b[j]$$

takes $\Theta(1)$.

Total time

$$n \cdot n \cdot \Theta(1) = \Theta(n^2).$$

Next question

Can we multiply polynomials faster than $\Theta(n^2)$?

Karatsuba Algorithm: Base Case (size = 1)

When do we stop recursing?

If each input has one digit (or one coefficient), do a direct multiplication.

$$a = [u], \quad b = [v] \implies c = [u \cdot v]$$

Example

$$a = [2], \quad b = [3] \implies c = [6]$$

Base-case rule

return $a[0] \cdot b[0]$

Karatsuba Algorithm: Recursive Case (size > 1)

Divide step

When the inputs are not in the base case, split each input into two halves:

$$a = a_H \cdot B^m + a_L, \quad b = b_H \cdot B^m + b_L$$

Here, a_H, b_H are the high halves, a_L, b_L are the low halves, and m is about half of the current size.

Input a

$$a \longrightarrow (a_H, a_L)$$

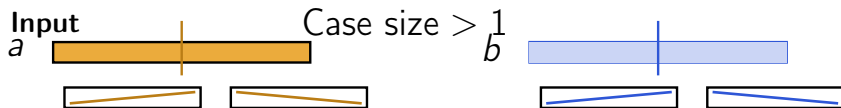
Input b

$$b \longrightarrow (b_H, b_L)$$

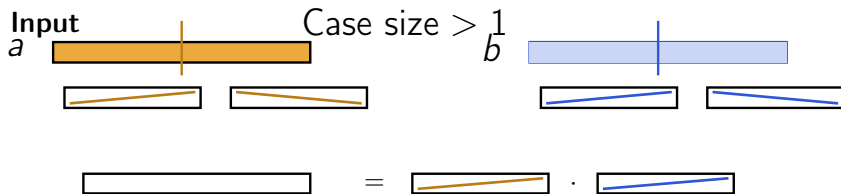
Decision rule

If size > 1, apply this split and continue recursively on smaller parts.

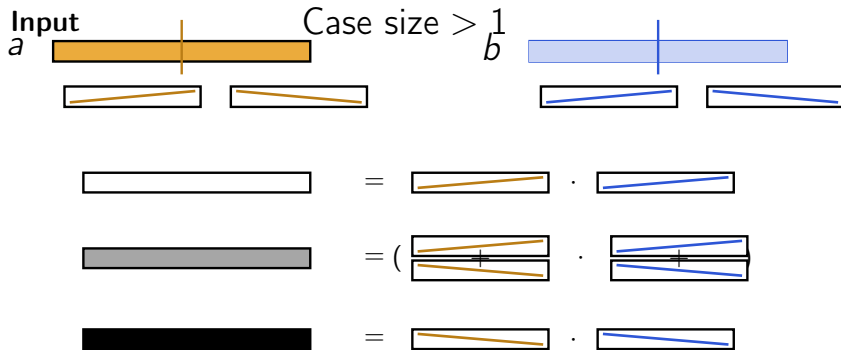
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



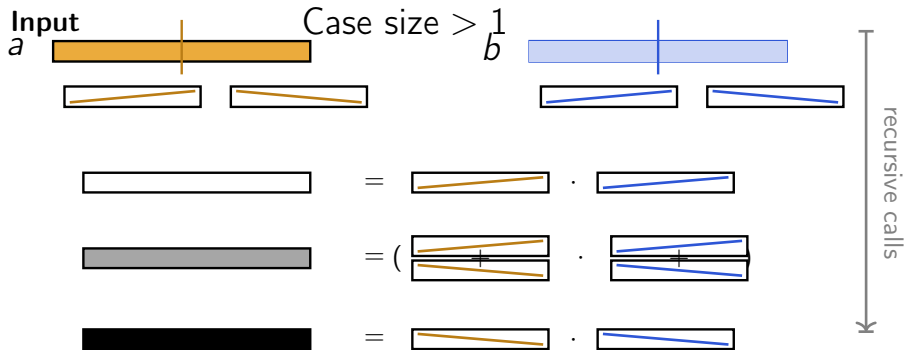
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



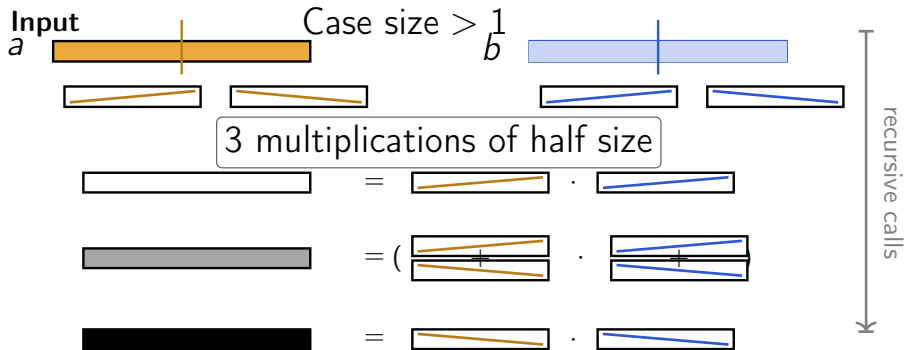
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



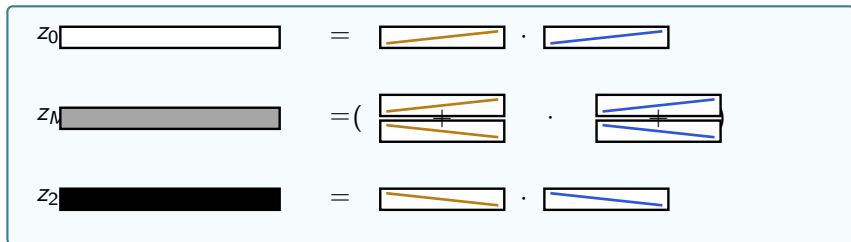
Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)



Karatsuba Algorithm: Combine the Three Products



Karatsuba Algorithm: Recursive Case ($\text{size} > 1$)

After splitting into halves

$$a = a_H \cdot B^m + a_L, \quad b = b_H \cdot B^m + b_L$$

compute only three products of half size:

$$z_2 = a_H b_H, \quad z_0 = a_L b_L, \quad z_M = (a_H + a_L)(b_H + b_L).$$

Why this is the key idea

Each of z_2, z_0, z_M is a recursive multiplication on smaller inputs (about half the size).

Recursive work per level

3 multiplications of half size

Karatsuba Algorithm: Combine the Three Products

Recover the middle term without a fourth multiplication

$$(a_H + a_L)(b_H + b_L) = a_H b_H + a_H b_L + a_L b_H + a_L b_L$$

so

$$a_H b_L + a_L b_H = z_M - z_2 - z_0.$$

Final recombination

$$ab = z_2 \cdot B^{2m} + (z_M - z_2 - z_0) \cdot B^m + z_0.$$

Decision rule

If size > 1: split, compute z_2, z_0, z_M recursively, then combine with the formula above.

Karatsuba Algorithm: Recursive Procedure

Input

Two length- n vectors (digits or coefficients):

$$a[0..n-1], \quad b[0..n-1].$$

Base case

If $n = 1$, return

$$a[0] \cdot b[0].$$

Recursive case ($n > 1$)

Let $s = \frac{n}{2}$, then split:

$$a_0 = a[0..s-1], \quad a_1 = a[s..n-1],$$

$$b_0 = b[0..s-1], \quad b_1 = b[s..n-1].$$

Three recursive multiplications

$$c_0 = \text{multiply}(a_0, b_0), \quad c_1 = \text{multiply}(a_0 + a_1, b_0 + b_1), \quad c_2 = \text{multiply}(a_1, b_1).$$

Karatsuba Algorithm: Recursive Procedure

Algorithm sketch

multiply($a[0..n-1]$, $b[0..n-1]$)

- If $n = 1$, return $a[0] \cdot b[0]$.
- Otherwise, let $s = \frac{n}{2}$ and split:

$$a_0 = a[0..s-1], \quad a_1 = a[s..n-1], \quad b_0 = b[0..s-1], \quad b_1 = b[s..n-1].$$

- Compute three recursive products:

$$c_0 = \text{multiply}(a_0, b_0), \quad c_1 = \text{multiply}(a_0 + a_1, b_0 + b_1), \quad c_2 = \text{multiply}(a_1, b_1).$$

- Return

$$c_0 + \text{shift}(c_1 - c_0 - c_2, s) + \text{shift}(c_2, n).$$

Karatsuba Algorithm: Running Time

Set up the recurrence

Let $T(n)$ be the running time for inputs of size n .

$$T(1) = \Theta(1)$$

For $n > 1$:

$$T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n).$$

Why this form?

- $3 T(n/2)$: three recursive multiplications on half-size inputs.
- $\Theta(n)$: additions, subtractions, splitting, and shifts.

Assumption

Assume $n = 2^d$, so each split is even.

Next step

How do we solve

$$T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n)?$$

Karatsuba Algorithm: Running Time (Expansion)

Recurrence

$$T(1) = \Theta(1), \quad T(n) = 3 T\left(\frac{n}{2}\right) + \Theta(n) \quad (n > 1).$$

Unroll a few steps

$$\begin{aligned} T(n) &= 3\left(3T\left(\frac{n}{4}\right) + \Theta\left(\frac{n}{2}\right)\right) + \Theta(n) \\ &= 3^2 T\left(\frac{n}{2^2}\right) + 3\Theta\left(\frac{n}{2}\right) + \Theta(n) \\ &\vdots \\ &= 3^d T\left(\frac{n}{2^d}\right) + \sum_{i=0}^{d-1} 3^i \Theta\left(\frac{n}{2^i}\right). \end{aligned}$$

Karatsuba Algorithm: Final Asymptotic Bound

Substitute $n = 2^d$

Because $\frac{n}{2^d} = 1$, we get

$$T(n) = 3^d \Theta(1) + \Theta\left(n \sum_{i=0}^{d-1} \left(\frac{3}{2}\right)^i\right).$$

The geometric sum is dominated by its largest term, so

$$T(n) = \Theta(3^d).$$

Rewrite in terms of n

$$3^d = 3^{\log_2 n} = n^{\log_2 3}.$$

Therefore,

$$T(n) = \Theta\left(n^{\log_2 3}\right).$$

Polynomial Multiplication: Comparing Methods

Asymptotic running times

- Trivial method: $\Theta(n^2)$
- Karatsuba: $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
- FFT-based methods: $O(n \log n)$

Asymptotic order (for large n)

$$n \log n \ll n^{1.585} \ll n^2.$$

At $n = 1024$

$$n^2 = 1,048,576 (> 10^6)$$

$$n^{1.585} \approx 5.9 \times 10^4 (< 60,000)$$

Caveat

Big- O hides constants, so asymptotically faster methods are not always best for very small inputs.

Practical Rule: Which Algorithm to Run?

Decision rule in practice

- If n is small, run the trivial $\Theta(n^2)$ algorithm.
- Otherwise, switch to Karatsuba.

Why this hybrid works

- Trivial method has low overhead and simple operations.
- Karatsuba wins after crossing a size threshold.
- The threshold depends on implementation and hardware.

Takeaway

small $n \Rightarrow$ trivial

larger $n \Rightarrow$ Karatsuba

FFT methods become preferable only for sufficiently large n .

Standard divide-and-conquer recurrence

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

- a : number of recursive subproblems,
- $\frac{n}{b}$: size of each subproblem,
- $f(n)$: non-recursive work (divide/combine).

Master Theorem: Important Caveats

Rounding Variant

The theorem also applies when $\frac{n}{b}$ is replaced by

$$\left\lfloor \frac{n}{b} \right\rfloor \quad \text{or} \quad \left\lceil \frac{n}{b} \right\rceil .$$

Constants Condition

a and b are constants (typically $a \geq 1$, $b > 1$).

Master Theorem: The Three Cases

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad c = \log_b a$$

Case A (leaf-dominated)

If $f(n) = O(n^{c-\varepsilon})$ for some $\varepsilon > 0$, then

$$T(n) = \Theta(n^c).$$

Case B (balanced)

If $f(n) = \Theta(n^c)$, then

$$T(n) = \Theta(n^c \log n).$$

Case C (root-dominated)

If $f(n) = \Omega(n^{c+\varepsilon})$ for some $\varepsilon > 0$, and (regularity condition)

$$a f\left(\frac{n}{b}\right) \leq k f(n) \quad \text{for some } k < 1,$$

then

$$T(n) = \Theta(f(n)).$$

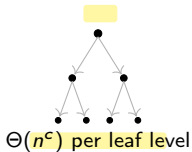
Master Theorem: Visual Intuition

Recursion tree viewpoint

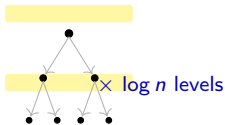
$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad c = \log_b a$$

At level i , there are a^i subproblems of size n/b^i , so the total work on that level is

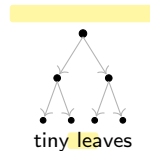
$$a^i f\left(\frac{n}{b^i}\right).$$



Case A
 $f(n)$ is smaller
than leaf work



Case B
every level contributes
about the same



Case C
the root dominates
all lower levels

Example: $T(n) = 3T(n/2) + \Theta(n)$

Match parameters

$$a = 3, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 3 \approx 1.58$$

Compare $f(n)$ with n^c

$$f(n) = \Theta(n) = \Theta(n^1)$$

So we can take $c' = 1$, and

$$1 < \log_2 3.$$

Apply Case 1

Since $f(n) = O(n^{c'})$ for $c' < c$, Case 1 applies.

$$T(n) = \Theta\left(n^{\log_2 3}\right).$$

Master Theorem Example: Binary Search

Recurrence

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

Match parameters

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

$$c = \log_b a = \log_2 1 = 0$$

Compare $f(n)$ with n^c

$$n^c = n^0 = 1$$

$$f(n) = \Theta(1) = \Theta(n^c)$$

So this is **Master Theorem Case 2**.

Conclusion

$$T(n) = \Theta(n^c \log n) = \Theta(n^0 \log n) = \Theta(\log n)$$

Merge Sort via Master Theorem

Recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

- $a = 2$
- $b = 2$
- $f(n) = \Theta(n)$
- $c = \log_b a = \log_2 2 = 1$
- Case A does not apply ($f(n)$ is not $O(n^{c'})$ for $c' < 1$).
- Case B applies since $f(n) = \Theta(n^c) = \Theta(n)$.

Conclusion

$$T(n) = \Theta(n^c \log n) = \Theta(n \log n).$$

Karatsuba Recurrence: already in Master Theorem form

Recurrence

$$T(n) = 3T(n/2) + \Theta(n)$$

Match to $T(n) = aT(n/b) + f(n)$

$$a = 3, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 3 \approx 1.585$$

Key comparison

$$f(n) = \Theta(n) = O(n^{c'}) \text{ for } c' = 1 < \log_2 3$$

So this is **Master Theorem Case A** (f is polynomially smaller than n^c).

Karatsuba memory complexity

Recurrence for extra memory

$$M(n) = M(n/2) + \Theta(n)$$

Match the Master Theorem

$$a = 1, \quad b = 2, \quad f(n) = \Theta(n)$$

$$c = \log_b a = \log_2 1 = 0$$

Check all cases

- Case A: no, because $f(n) = \Theta(n)$ is not $O(n^{c'})$ for any $c' < 0$.
- Case B: no, because $f(n) \neq \Theta(n^c) = \Theta(1)$.
- Case C: yes, since $f(n) = \Theta(n) = \Omega(n^{c'})$ for $c' = 1 > 0 = c$.

Conclusion

$$M(n) = \Theta(f(n)) = \Theta(n).$$

Algorithms for Data Science

Lecture #5: Sorting

Mateusz Buczyński

University of Warsaw

25.03.2026

Thanks to dr Krzysztof Fleszar for the base material.

Sorting: Problem Statement

Input

- An array $a[1..n]$

Output

- The array is sorted in nondecreasing order:

$$a[1] \leq a[2] \leq \dots \leq a[n]$$

Practical note

- Sorting is needed very often.
- In practice, a sorting routine is usually already implemented (library function).

A Surprising Fact About Sorting

Theorem (comparison model)

Any comparison-based sorting algorithm requires

$$\Omega(n \log n)$$

comparisons in the worst case.

Important caveat

This lower bound applies only to algorithms that learn order *through comparisons* (e.g., “ $a_i < a_j$ ”).

- So no comparison sort can be worst-case $O(n)$ or $O(n \log \log n)$.
- Efficient comparison sorts (merge sort, heap sort) are asymptotically optimal.

Why $\Omega(n \log n)$?

Setup (assume all elements are distinct)

- There are $n!$ possible input orders (permutations).
- Exactly one permutation is the correct sorted order for a given input.

Key counting idea

- One comparison has 2 outcomes, so it can eliminate at most about half of the remaining possibilities.
- After k comparisons, at most 2^k cases can be distinguished.
- To distinguish all $n!$ cases, we need

$$2^k \geq n! \Rightarrow k \geq \log_2(n!).$$

Conclusion

$$k \geq \log_2(n!) \approx \log_2(n^n) = n \log_2 n \Rightarrow k = \Omega(n \log n).$$

Insertion Sort: Visual Idea

Grow a sorted prefix

At each step, take the next element and insert it into the already sorted left part.

Example

[4 | 1, 5, 3]

[1, 4 | 5, 3]

[1, 4, 5 | 3]

[1, 3, 4, 5]

What to notice

- Left side is always sorted.
- One new element is inserted each round.
- Good intuition: like sorting cards in your hand.

Insertion Sort: One Insertion Step

Input example

$$a = [4, 1, 5, 3, 6, 2]$$

Teaching idea

At this moment of insertion sort, we treat the last value as the **key** and insert it into the already sorted prefix.

Decision rule

For $\text{key} = 2$, scan from right to left in the sorted part:

$$[1, 3, 4, 5, 6]$$

Shift every element > 2 one position to the right, then place 2 into the first free position.

Insertion Sort: Result of Inserting 2

Before insertion

[1, 3, 4, 5, 6, 2]

After insertion

[1, 2, 3, 4, 5, 6]

What changed?

- 6, 5, 4, 3 were shifted one cell to the right.
- 2 was inserted between 1 and 3.

Insertion Sort: Complexity at a Glance

Core properties

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: **Yes**

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: **Yes**

Best case

- Input already sorted
- Time: $\Theta(n)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: **Yes**

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \dots + n$$

- Time:

$$\Theta(1 + 2 + \dots + n) = \Theta(n^2)$$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: **Yes**

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \dots + n$$

- Time:

$$\Theta(1 + 2 + \dots + n) = \Theta(n^2)$$

Average case

$$\Theta(n^2)$$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: **Yes**

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \dots + n$$

- Time:

$$\Theta(1 + 2 + \dots + n) = \Theta(n^2)$$

Average case

$$\Theta(n^2)$$

Practical note

Insertion sort is often fast in practice for small n .

Why “Stable” Matters

Definition (stability)

A sorting algorithm is **stable** if equal elements keep their original relative order.

Input with labels

$[1_A, 2, 1_B]$

(1_A appears before 1_B)

After stable sort

$[1_A, 1_B, 2]$

Order of equal keys is preserved.

If not stable

$[1_B, 1_A, 2]$

This changes the order of equal keys.

Insertion sort

Insertion sort is stable, because insertion shifts larger elements right and does not swap equal keys past each other.

Merge Sort: Visual Idea

Split, sort, merge

Break the array into smaller parts, sort them recursively, then merge sorted parts back together.

Example

[4, 1, 5, 3]
[4, 1] [5, 3]
[1, 4] [3, 5]
[1, 3, 4, 5]

What to notice

- The problem is divided into smaller subproblems.
- Merging two sorted lists is linear.
- The structure is very regular.

Merge Sort: Idea and Workflow

Core properties

- Paradigm: **divide & conquer**
- Stable:

Merge Sort: Idea and Workflow

Core properties

- Paradigm: **divide & conquer**
- Stable: **Yes** (equal elements keep relative order)
- Extra space:

Merge Sort: Idea and Workflow

Core properties

- Paradigm: **divide & conquer**
- Stable: **Yes** (equal elements keep relative order)
- Extra space: $O(n)$

Merge Sort: Running Time

Recurrence

For $n > 1$, merge sort performs:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

- $2T(n/2)$: recursively sort two halves
- $\Theta(n)$: merge step

Best case = Worst case

- The algorithm always splits into halves the same way.
- Merging still scans all elements, regardless of initial order.

$$T(n) = \Theta(n \log n)$$

At a glance

Time: $\Theta(n \log n)$, Space: $O(n)$, Stable: Yes

Quick Sort: Visual Idea

Partition around a pivot

Choose one element as a pivot, place smaller elements on one side and larger elements on the other, then recurse.

Example

$[4, 1, 5, 3]$
 $[1, 3] < 4 < [5]$
 $[1, 3] \rightarrow [1, 3]$
 $[1, 3, 4, 5]$

What to notice

- The pivot lands in its final position.
- Only the left and right parts remain to be sorted.
- Performance depends on partition balance.

Practical note

Quick sort is often very fast in practice for large n .

Quick Sort: Complexity and Caveats

At a glance

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Extra space: $O(\log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Extra space: $O(\log n)$

Stable: No

Why worst case can be quadratic

If partitioning is very unbalanced (e.g., pivot always smallest/largest), recursion depth becomes n :

$$T(n) = T(n-1) + \Theta(n)$$

$$\Rightarrow \Theta(n + (n-1) + \dots + 1) = \Theta(n^2).$$

Typical bad pattern

With pivot = first element and reverse input

$[7, 6, 5, 4, 3, 2, 1],$

each step removes only one element

Data Structures for Dynamic Sets

Notation

n = current number of stored elements.

Why structure matters

A data structure keeps an **invariant** (or not), and that directly determines operation costs.

Structure	add	find	find max	remove max
Unsorted array (no invariant)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Binary heap (heap property)	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$
Sorted array (sorted invariant)	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(1)$

Takeaway

There is no universally best structure: performance depends on which operations are most frequent.

How the Invariant Changes the Cost

Unsorted array

- Add at end: $\Theta(1)$
- To find max, scan all: $\Theta(n)$
- Remove max needs scan first

Binary heap

- Max kept at root: $\Theta(1)$
- Add/remove via sift up/down: $\Theta(\log n)$
- Arbitrary find still $\Theta(n)$

Sorted array

- Binary search: $\Theta(\log n)$
- Max at one end: $\Theta(1)$
- Add needs shifts: $\Theta(n)$

Decision rule

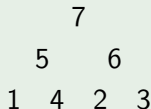
If you need fast find max and remove max with many updates, a **binary heap** is usually the best compromise.

What Does a Heap Look Like?

Binary heap = complete binary tree

It is usually stored in an array, but we can visualize it as a nearly full binary tree.

Tree view (max-heap)



Each parent is at least as large as its children.

Array view

[7, 5, 6, 1, 4, 2, 3]

- Root at index 1
- Children of index k : $2k$ and $2k + 1$
- Parent of index k : $\lfloor k/2 \rfloor$

Important

A heap is **not** a sorted array. It guarantees only local order: parent vs. children.

Heap Sort via a Binary Heap

Key idea

Maintain a **max-heap** as the data structure.

Invariant

Heap property: every parent key is $\geq$ each child key.

- Let n be the **maximum number of elements** stored.
- This invariant is what makes operations efficient.

Binary Heap Operation Costs

Running times

- **add:** $O(\log n)$
- **find max:** $\Theta(1)$
- **remove max:** $O(\log n)$

Why this is useful for heap sort

Sorting repeatedly uses **remove max**, so we rely on the $O(\log n)$ bound while max lookup stays constant time.

Heap Sort: Idea and Heap Operations

Invariant

Heap property is maintained at all times (max-heap).

Binary Heap Operations

- $\text{add}(x)$: $O(\log n)$
- $\text{findMax}()$: $\Theta(1)$
- $\text{removeMax}()$: $O(\log n)$

Heap Sort Strategy

- Build a max-heap from all input elements.
- Repeatedly extract maximum.
- Write extracted values from right to left.

Heap Sort Algorithm and Cost

Input / Output

Input: array $a[1..n]$

Output: a is sorted (nondecreasing order)

Algorithm (using auxiliary heap H)

$H \leftarrow$ new empty max-heap

for $i = 1$ to n : $H.add(a[i])$

for $i = n$ to 1 : $a[i] \leftarrow H.findMax(), H.removeMax()$

Running Time

- Build phase: $n \cdot O(\log n) = O(n \log n)$
- Extract phase: $n \cdot (\Theta(1) + O(\log n)) = O(n \log n)$
- Total: $\Theta(n \log n)$ (best and worst case)

Properties

- Extra space: $\Theta(n)$
- Stable: No

Heap Sort Optimization: Build Heap In-Place

Main idea

Do not create an auxiliary heap. Treat the input array itself as a max-heap.

In-place initialization

$$H \leftarrow \text{make_heap}(a)$$

Time: $\Theta(n)$

Why this matters

- We avoid storing all elements in a separate heap.
- Heap construction is linear, not $n \cdot O(\log n)$.
- The heap invariant is preserved on the active prefix of the array.

Optimized Heap Sort: Algorithm and Final Costs

Algorithm (in-place)

Input: array $a[1..n]$

Output: a sorted in nondecreasing order

$H \leftarrow \text{make_heap}(a)$

for $i = n$ downto 1 : $H.\text{removeMax}()$

Running time

- Build heap: $\Theta(n)$
- n extractions: $n \cdot O(\log n) = O(n \log n)$
- Total: $\Theta(n \log n)$ (best case = worst case)

Properties

- Extra space: $\Theta(1)$
- Stable: No

Comparison and Practice

Stable sorting

A sorting algorithm is **stable** if equal elements remain in their original relative order.

Algorithm	Time	Extra space	Stable
Insertion Sort	$\Theta(n^2)$	$\Theta(1)$	Yes
Merge Sort	$\Theta(n \log n)$	$\Theta(n)$	Yes
Quick Sort	$\Theta(n^2)$ worst case	$\Theta(\log n)$	No
Heap Sort	$\Theta(n \log n)$	$\Theta(1)$	No

Practical reading of the table

- Insertion Sort is simple and good for small inputs.
- Merge Sort gives predictable $\Theta(n \log n)$ time and stability.
- Quick Sort is often very fast in practice, but has a quadratic worst case.
- Heap Sort keeps $\Theta(n \log n)$ time with only constant extra memory.

Heap Representation in an Array

One-based indexing (max-heap stored in $a[1..i]$)

$a[1]$ $a[2]$ $a[3]$ $a[4]$ $a[5]$ $a[6]$ $a[7]$ ($i = 7$)

Same structure as a binary tree

index 1

indices 2, 3

indices 4, 5, 6, 7

- Root is at index 1.
- Children of index 2: 4, 5.
- Children of index 3: 6, 7.

Why this is useful

No pointers are needed:

- Tree navigation is done by index arithmetic.
- Heap operations (add, removeMax) move along parent/child indices.

Heap Index Formulas (1-Based)

For a node at index k

$$\text{parent}(k) = \left\lfloor \frac{k}{2} \right\rfloor, \quad \text{left}(k) = 2k, \quad \text{right}(k) = 2k + 1$$

Concrete example: $k = 3$

$$\text{parent}(3) = 1$$

$$\text{left}(3) = 6$$

$$\text{right}(3) = 7$$

Boundary rule

With current heap size i :

- A child exists only if its index is $\leq i$.
- If $2k > i$, node k is a leaf.

Heap Example: Valid Case

Max-Heap Property (array view)

For all indices k with $1 < k \leq i$:

$$a[k] \leq a[\lfloor k/2 \rfloor]$$

Example a with $i = 7$

$$a[1..7] = [7, 5, 6, 1, 4, 2, 3]$$

- Parent of index k is at $\lfloor k/2 \rfloor$.
- Every child value is $\leq$ its parent value.

So this is a heap.

Heap Example: Non-Heap Case

Counterexample b

$$b[1..7] = [7, 5, 4, 6, 3, 2, 1]$$

Check index $k = 4$:

$$b[4] = 6, \quad \lfloor 4/2 \rfloor = 2, \quad b[2] = 5$$

But $6 \not\leq 5$, so the heap property is violated.

Decision Rule

A max-heap requires:

$$\forall k (1 < k \leq i) : a[k] \leq a[\lfloor k/2 \rfloor]$$

If one index fails, it is not a heap.

Heap Example: Active Heap Prefix

Max-heap property (1-based indexing)

For all indices k with $1 < k \leq i$:

$$a[k] \leq a\left[\left\lfloor \frac{k}{2} \right\rfloor\right]$$

Given array and heap size

$$a[1..7] = [6, 5, 4, 3, 2, 1, 99], \quad i = 6$$

- The current heap is only $a[1..i] = a[1..6]$.
- $a[7] = 99$ is outside the active heap.

Why This Is Still a Heap

Tree view of $a[1..6]$

```
      6
     / \
    5   4
   / \ / \
  3 2 1
```

Checks ($1 < k \leq 6$)

$5 \leq 6, \quad 4 \leq 6,$
 $3 \leq 5, \quad 2 \leq 5, \quad 1 \leq 4$

Decision

Yes, this is a heap for $i = 6$. Only indices $1..i$ are tested.

Heap: Operations (Max-Heap)

Heap property

For every node except the root:

$$a[k] \leq a\left[\left\lfloor \frac{k}{2} \right\rfloor\right], \quad 1 < k \leq i$$

So each child is at most its parent.

Array view (example)

index	1	2	3	4	5	6
$a[\text{index}]$	6	5	4	3	1	9

Indices map to a complete binary tree: parent of k is $\lfloor k/2 \rfloor$, children are $2k$ and $2k + 1$.

Find maximum

In a max-heap, the maximum is always at the root:

```
return a[1]
```


Heap Operations: Add and Remove Max

Add(v) (sift up)

- Put v in the next free slot:
 $a[i + 1] \leftarrow v$, then $i \leftarrow i + 1$.
- While parent is smaller, swap with parent.
- Stop when heap property is restored.

Key idea

Insertion may move the new value upward until

$$a[\text{parent}] \geq a[\text{child}].$$

Remove Max (sift down)

- Swap $a[1]$ with $a[i]$ (move max to the end).
- Decrease heap size: $i \leftarrow i - 1$.
- Starting at $a[1]$, while a larger child exists:
 - swap with the larger child.
- Stop when heap property is restored.

Max-Heap Operations: Find Max and Add

Find Max

In a max-heap, the largest element is at the root:

return $a[1]$

$$T(n) = \Theta(1)$$

Add(v)

- Insert at the end:
 $a[i + 1] \leftarrow v, i \leftarrow i + 1$.
- Restore heap order by **Upheap(j)**:
 - while parent is smaller, swap with parent.

Upheap(j) idea

- Node moves only along the path to the root.
- At most one swap per level.

Cost

$$T_{\text{add}}(n) = T_{\text{upheap}}(n) = O(\log n)$$

Max-Heap Operation: Remove Max

Remove Max

- Swap $a[1]$ with $a[i]$ (max goes to the end).
- Decrease heap size: $i \leftarrow i - 1$.
- Starting from root, run **Downheap(j)**:
 - while a larger child exists, swap with the larger child.

Downheap(j) cost

$$T_{\text{removeMax}}(n) = T_{\text{downheap}}(n) = O(\log n)$$

Why $O(\log n)$?

- A binary heap is a complete binary tree.
- With i elements, tree height is $O(\log i)$.
- Upheap/Downheap follow one root-to-leaf path.

Notation note

n = current number of heap elements (here: $n = i$).

Breaking the $\Omega(n \log n)$ Bound

Comparison-based sorting lower bound

If an algorithm relies only on comparisons, then its worst-case time is

$$\Omega(n \log n).$$

Counting Sort: when we can do better

Input: array $a[1..n]$ such that

$$0 \leq a[i] \leq k \quad \text{for all } i.$$

Assumption: k is known (or precomputed).

Counting Sort: Core Procedure

High-level steps

- Count how often each value $0, 1, \dots, k$ appears.
- Compute cumulative sums (prefix counts).
- Place each element into its correct final position.

Radix Sort: Breaking the Comparison Bound

Key Idea

Use a **stable** linear-time sort (e.g., Counting Sort) multiple times on different digits/fields.

Time per pass: $\Theta(n + k)$

Example: Sort students by birthdate

Represent each date as `Year.Month.Day`.

Algorithm (LSD $\rightarrow$ MSD)

- Sort by **Day**
- Then sort by **Month**
- Then sort by **Year**

Typical key ranges

$\text{Day} \leq 31, \quad \text{Month} \leq 12, \quad \text{Year} \leq 2026$

(so each pass can use Counting Sort)

Why This Order Is Correct

Crucial property

Counting Sort is **stable**: equal keys keep their previous relative order.

- After sorting by Day, ties are unresolved by Month/Year.
- Sorting by Month keeps Day-order inside each month.
- Sorting by Year keeps Month+Day order inside each year.

Important caveat

Sorting in the opposite order (**Year** $\rightarrow$ **Month** $\rightarrow$ **Day**) does **not** produce correct full-date order.

Stalin Sort: A Joke Counterexample

Idea

Scan from left to right and keep only elements that are at least as large as the last kept one. All other elements are simply **removed**.

Example

$[3, 1, 4, 2, 5] \Rightarrow [3, 4, 5]$

because 1 and 2 are “purged”.

Why this is not a real sorting algorithm

- Output may have fewer elements than input.
- It does not produce a permutation of the original array.
- So it solves a different problem.

Teaching note

Like Bogosort, Stalin Sort is mainly a humorous example used to discuss what a sorting



Order Statistics: QuickSelect

Problem

Instead of sorting everything, we may want only the **k -th smallest element**: minimum, maximum, median, quartile, etc.

Key idea

QuickSelect uses the same **partition** step as Quick Sort. After partitioning around a pivot, only **one side** can still contain the desired rank.

Consequence

We recurse into only one subproblem, not both.

QuickSelect: Cost and Median

Running time

Best / average: $\Theta(n)$

Worst case: $\Theta(n^2)$

The worst case appears when pivots are repeatedly very unbalanced.

Median

To find the median, ask for rank

$$k = \left\lceil \frac{n}{2} \right\rceil.$$

So the median can usually be found without fully sorting the array.

Comparison with sorting

If we only need one order statistic, QuickSelect is typically better than paying $\Theta(n \log n)$ for a full sort.

External Sorting

When do we need it?

Sometimes the dataset is so large that it does not fit into RAM. Then the main bottleneck is often **disk I/O**, not CPU time.

Basic strategy

- Load a chunk that fits into memory.
- Sort it internally and write it back to disk as a **run**.
- Repeat for all chunks.
- Merge the sorted runs.

Main idea

External Sorting is essentially **Merge Sort adapted to secondary storage**.

External Sorting: Why Merging Works Well

Why merge?

- Runs are already sorted.
- Sequential disk access is much faster than many random reads.
- Multi-way merge can combine many runs at once.

Typical tools

- Buffer pages in RAM
- Priority queue / heap
- Temporary files on disk

Takeaway

For massive datasets, the right question is not only “How many comparisons?”, but also “How many passes over disk data?”

Algorithms for Data Science

Lecture #6: Dynamic Programming

Mateusz Buczynski

University of Warsaw

15.04.2026

Thanks to dr Krzysztof Fleszar for the base material.

Dynamic Programming: The Core Idea

When DP is useful

Build the answer to a hard problem from answers to smaller subproblems.

Two signals

- Overlapping subproblems
- Optimal substructure

Main benefit

Do not solve the same subproblem again and again.

Typical DP Workflow

- 1 Define a **state**: what subproblem does one table entry mean?
- 2 Write a **recurrence**: how does this state depend on smaller states?
- 3 Specify **base cases**.
- 4 Choose an evaluation order or use memoization.
- 5 Read off the final answer, and sometimes reconstruct the solution.

Teaching rule

If the state is vague, the DP usually goes wrong.

Warm-up Example: Fibonacci Numbers

Definition

$$F_1 = 1, \quad F_2 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ for } n \geq 3$$

First values

1, 1, 2, 3, 5, 8, 13, ...

Task

Given n , compute F_n .

Naive Recursion

Direct translation of the definition

$$\text{fib}(n) = \begin{cases} 1, & n \leq 2, \\ \text{fib}(n-1) + \text{fib}(n-2), & n > 2. \end{cases}$$

Why it is attractive

The code is short and the logic matches the recurrence exactly.

Problem

The same values such as $\text{fib}(5)$ and $\text{fib}(4)$ are recomputed many times.

Fibonacci via Naive Recursion

Pseudocode

```
function fib(n):  
    if n <= 2:  
        return 1  
  
    return fib(n - 1) + fib(n - 2)
```

Cost

Simple, but exponential because the same subproblems repeat.

Why Naive Recursion Is Slow

Time recurrence

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

Consequence

The recursion tree grows exponentially:

$$T(n) = \Theta(\varphi^n)$$

where $\varphi \approx 1.618$.

Key lesson

Short code is not the same as efficient code.

Memoization: Top-Down DP

Idea

Store each computed value the first time it appears, then reuse it.

Memoization pattern

- if n is already in memo, return it
- otherwise compute it once
- save it in memo

Effect

Each Fibonacci number is computed at most once.

Memoized Fibonacci Algorithm

Pseudocode

```
memo = empty dictionary
function fib(n):
    if n in memo:
        return memo[n]
    if n <= 2:
        memo[n] = 1
    else:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

Cost

Each state $1, 2, \dots, n$ is computed once, so the running time is linear.

Memoized Fibonacci: Complexity

State space

The subproblems are $1, 2, \dots, n$, so there are only n distinct states.

Average-case view with dictionary lookups

$$\text{Time} = \Theta(n), \quad \text{Space} = \Theta(n)$$

Practical view

Much faster than exponential recursion, but still uses recursion depth and cache memory.

Bottom-Up DP: Tabulation

Alternative viewpoint

Instead of asking for F_n and recursively going down, fill the answers in increasing order.

Table order

$$F_1, F_2, F_3, \dots, F_n$$

with

$$F_i = F_{i-1} + F_{i-2}.$$

Main advantage

No recursion overhead.

Bottom-Up Fibonacci Algorithm

Pseudocode

```
function fib(n):  
    if n <= 2:  
        return 1  
    F[1] = 1  
    F[2] = 1  
    for i = 3 to n:  
        F[i] = F[i - 1] + F[i - 2]  
    return F[n]
```

Cost

Iterative evaluation gives $\Theta(n)$ time with no recursion stack.

Top-Down vs Bottom-Up

Top-down

- Start from the final question
- Compute only visited states
- Natural when recurrence comes first

Bottom-up

- Start from base cases
- Fill the table in a valid order
- Often simpler and faster in practice

Decision rule

If there is a clear order on the states, bottom-up is often the cleaner implementation.

Can We Use Less Space?

Observation

To compute F_i , we only need the two previous values.

Space optimization

Keep only

$$\text{prev2} = F_{i-2}, \quad \text{prev1} = F_{i-1}.$$

Then compute

$$\text{curr} = \text{prev1} + \text{prev2}.$$

Result

$$\text{Time} = \Theta(n), \quad \text{Space} = \Theta(1)$$

A Useful Warning: Bit Complexity

What we counted so far

We treated one arithmetic operation as $\Theta(1)$.

But Fibonacci numbers grow

The number of bits in F_n is $\Theta(n)$, so adding big integers is not truly constant-time.

Takeaway

Algorithm analysis depends on the cost model. For basic DP design, the RAM model is fine; for exact arithmetic, bit complexity can matter.

What Fibonacci Teaches About DP

- 1 Identify repeated subproblems.
- 2 Store the answers.
- 3 Reuse them instead of branching exponentially.

But

Fibonacci is too easy. Real DP becomes interesting when the state has more than one dimension.

Memoization vs. Bottom-Up DP

Often interchangeable in principle.

Recursion with Memoization

- recursive
- top-down
- stores answers on demand
- may pay recursion overhead (slower)
- limited by recursion depth (risk)
- often easier to write (simpler)

Useful when the recurrence is natural
and not all states are needed.

Dynamic Programming

- iterative
- bottom-up
- fills a table in dependency order
- usually uses arrays (faster)
- no recursion stack (safe)
- evaluation order can be less obvious (harder)

Useful when the table structure is clear
and many states must be computed
anyway.

0/1 Knapsack: Problem Statement

Input

Items $1, \dots, n$, each with weight w_i and value v_i , and capacity M .

Goal

Choose a subset of items of total weight at most M with maximum total value.

Why this is a classic DP problem

Every item creates a binary decision: take it or skip it.

Optimization Formulation

Decision variables

$$x_i = \begin{cases} 1, & \text{if item } i \text{ is taken,} \\ 0, & \text{otherwise.} \end{cases}$$

Objective and constraint

$$\max \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq M$$

Why Simple Greedy Rules Fail

A tempting idea

Take the item with the best value or best value-to-weight ratio first.

Problem

For 0/1 knapsack, a locally good choice can block a much better combination later.

Teaching point

Greedy works for some optimization problems, but not for all of them.

Greedy Counterexample

Instance

$$M = 100$$

$$(v_1, w_1) = (2, 100), (v_2, w_2) = (1, 1), \dots, (v_{100}, w_{100}) = (1, 1)$$

Greedy by density

Take item 1 first, then no remaining item fits.

Greedy value = 2

Better choice

Take items 2 to 100:

Optimum = 99

Greedy Counterexample #2

Instance

$$M = 100$$

$$(v_1, w_1) = (52, 51), (v_2, w_2) = (50, 50), (v_3, w_3) = (50, 50)$$

Greedy by density

Take item 1 first, then no remaining item fits.

Greedy value = 52

Better choice

Take items 2 and 3:

Optimum = 100

Backtracking State

State definition

$\text{pack}(m, k) = \text{best value using capacity } m \text{ and items } 1, \dots, k$

Two choices at item k

- skip item k
- take item k , if $w_k \leq m$

Knapsack Recurrence

Recursive definition

$$\text{pack}(m, k) = \begin{cases} 0, & k = 0, \\ \text{pack}(m, k - 1), & w_k > m, \\ \max(\text{pack}(m, k - 1), v_k + \text{pack}(m - w_k, k - 1)), & w_k \leq m. \end{cases}$$

Naive complexity

$$\Theta(2^n)$$

because the recursion branches on many subsets.

Knapsack via Naive Recursion

Pseudocode

```
function pack(m, k):  
    if k == 0:  
        return 0  
    if w[k] > m:  
        return pack(m, k - 1)  
    return max(pack(m, k - 1),  
               v[k] + pack(m - w[k], k - 1))
```

Cost

The recursion explores many subsets, so the running time is exponential.

Memoization for Knapsack

State space

The state is a pair (m, k) , not just one number.

How many states?

$$(M + 1)(n + 1)$$

possible pairs.

Effect

Each state is solved at most once.

Knapsack via Recursion with Memoization

Complexity

At most $(M + 1)(n + 1)$ states, so time and space are both $\Theta(Mn)$.

Pseudocode

```
memo = new dictionary
function pack(m, k):
    if (m, k) not in memo:
        if k == 0:
            memo[(m, k)] = 0
        else if w[k] > m:
            memo[(m, k)] = pack(m, k - 1)
        else:
            memo[(m, k)] = max(pack(m, k - 1),
                               v[k] + pack(m - w[k], k - 1))
    return memo[(m, k)]
return pack(M, n)
```

Bottom-Up Table for Knapsack

DP table

Create $DP[0..n][0..M]$, where

$DP[k][m]$ = best value using items $1, \dots, k$ and capacity m .

Base row

$$DP[0][m] = 0 \quad \text{for all } m$$

Bottom-Up Knapsack Algorithm

Pseudocode

```
for m = 0 to M:
    DP[0][m] = 0
for k = 1 to n:
    for m = 0 to M:
        if w[k] > m:
            DP[k][m] = DP[k - 1][m]
        else:
            DP[k][m] = max(DP[k - 1][m],
                           v[k] + DP[k - 1][m - w[k]])
return DP[n][M]
```

Cost

The table has $(n + 1)(M + 1)$ entries, so time and space are both $\Theta(nM)$.

Knapsack Transition

If item k does not fit

$$DP[k][m] = DP[k - 1][m]$$

If item k fits

$$DP[k][m] = \max(DP[k - 1][m], v_k + DP[k - 1][m - w_k])$$

Interpretation

Either the optimal solution for this state uses item k , or it does not.

Knapsack Table-Filling Order

A valid order

Fill rows for

$$k = 1, 2, \dots, n$$

and inside each row fill capacities

$$m = 0, 1, \dots, M.$$

Why this works

Every state only depends on the previous row.

Knapsack Complexity

Time

$$\Theta(nM)$$

Space

$$\Theta(nM)$$

for the full table.

Important caveat

This is polynomial in n and M , but not polynomial in the bit-length of the input when M is large.

Recovering the Chosen Items

The DP table gives the value

$$DP[n][M]$$

but we often want the actual chosen subset too.

Traceback idea

Start from (n, M) and compare

$$DP[k][m] \text{ with } DP[k-1][m].$$

If they are equal, item k was skipped. Otherwise it was taken.

Knapsack Space Optimization

Observation

Each row depends only on the previous row.

Optimization

Use only one array $DP[0..M]$, but update capacities in decreasing order.

Why decreasing order matters

It prevents one item from being used multiple times in the same iteration.

What Knapsack Adds to the DP Toolbox

- multidimensional states
- decision-based recurrence
- table reconstruction
- space optimization by reusing rows

Pattern

Many DP problems are really about making one local decision per state.

Comparing Two Sequences

Motivating question

How similar are two strings, two code versions, or two DNA fragments?

One natural notion

Look for a long sequence of symbols that appears in both inputs in the same order.

Longest Common Subsequence (LCS)

Definition

A **subsequence** may skip symbols, but must preserve order.

Example

For

$a = \text{ABCB DAB}, \quad b = \text{BDCABA}$

one common subsequence is

BCBA.

Task

Find the maximum possible length.

LCS State

State definition

$L(i, j)$ = length of the LCS of prefixes $a[1..i]$ and $b[1..j]$

Why prefixes?

They make the recurrence local: each decision only looks at the last characters.

LCS Recurrence

Case 1: last characters match

If $a_i = b_j$, then

$$L(i, j) = L(i - 1, j - 1) + 1.$$

Case 2: last characters differ

If $a_i \neq b_j$, then

$$L(i, j) = \max(L(i - 1, j), L(i, j - 1)).$$

LCS Base Cases

If one prefix is empty

$$L(i, 0) = 0, \quad L(0, j) = 0$$

Interpretation

An empty string has no non-empty common subsequence with anything.

LCS via Naive Recursion

Pseudocode

```
function L(i, j):  
    if i == 0 or j == 0:  
        return 0  
    if a[i] == b[j]:  
        return L(i - 1, j - 1) + 1  
    return max(L(i - 1, j), L(i, j - 1))
```

Cost

Without caching, many prefix pairs (i, j) are recomputed.

Memoization for LCS

Pseudocode

```
memo = empty dictionary
function L(i, j):
    if (i, j) in memo:
        return memo[(i, j)]
    if i == 0 or j == 0:
        memo[(i, j)] = 0
    else if a[i] == b[j]:
        memo[(i, j)] = L(i - 1, j - 1) + 1
    else:
        memo[(i, j)] = max(L(i - 1, j), L(i, j - 1))
    return memo[(i, j)]
return L(n, m)
```

Cost

There are only $(n + 1)(m + 1)$ states, so memoization makes the algorithm $\Theta(nm)$.

LCS Table and Complexity

Table size

$$(n + 1)(m + 1)$$

for strings of lengths n and m .

Time

$$\Theta(nm)$$

Space

$$\Theta(nm)$$

for the full table.

Bottom-Up LCS Algorithm

Pseudocode

```
for i = 0 to n:  
    DP[i][0] = 0  
for j = 0 to m:  
    DP[0][j] = 0  
for i = 1 to n:  
    for j = 1 to m:  
        if a[i] == b[j]:  
            DP[i][j] = DP[i - 1][j - 1] + 1  
        else:  
            DP[i][j] = max(DP[i - 1][j], DP[i][j - 1])  
return DP[n][m]
```

Cost

Bottom-up evaluation visits each prefix pair once.

Recovering the Actual LCS

Traceback rule

Start from (n, m) :

- if $a_i = b_j$, that character belongs to the answer
- otherwise move to the larger of $L(i-1, j)$ and $L(i, j-1)$

Result

The table can give both the length and the sequence itself.

Why LCS Is Useful

- file comparison and version control
- plagiarism detection signals
- computational biology
- sequence alignment preprocessing

Teaching point

DP is not only about numbers; it is also about structured comparisons.

Levenshtein Distance (Edit Distance)

Question

How many single-character edits are needed to transform one string into another?

Allowed operations

- insert one character
- delete one character
- replace one character

Example

kitten → sitting has edit distance 3.

Why Edit Distance Matters

- spell checking
- fuzzy matching in databases
- OCR and speech-recognition cleanup
- bioinformatics and approximate sequence matching

Difference from LCS

LCS measures the longest preserved order; edit distance measures the minimum transformation cost.

Edit Distance State

State definition

$D(i, j)$ = minimum edit distance between prefixes $a[1..i]$ and $b[1..j]$

Again

Prefixes create smaller subproblems with a clear table structure.

Edit Distance Base Cases

If one prefix is empty

$$D(i, 0) = i, \quad D(0, j) = j$$

Interpretation

To turn a length- i string into an empty string, delete i characters. To go the other way, insert j characters.

Edit Distance Recurrence

If the last characters already match

If $a_i = b_j$, then

$$D(i, j) = D(i - 1, j - 1).$$

Otherwise

$$D(i, j) = 1 + \min (D(i - 1, j), D(i, j - 1), D(i - 1, j - 1))$$

for delete, insert, replace.

Levenshtein via Naive Recursion

Pseudocode

```
function D(i, j):  
    if i == 0:  
        return j  
    if j == 0:  
        return i  
    if a[i] == b[j]:  
        return D(i - 1, j - 1)  
    return 1 + min(D(i - 1, j),  
                   D(i, j - 1),  
                   D(i - 1, j - 1))
```

Cost

The branching is large, so naive recursion quickly becomes impractical.

Memoization for Levenshtein Distance

Pseudocode

```
memo = empty dictionary
function D(i, j):
    if (i, j) in memo:
        return memo[(i, j)]
    if i == 0:
        memo[(i, j)] = j
    else if j == 0:
        memo[(i, j)] = i
    else if a[i] == b[j]:
        memo[(i, j)] = D(i - 1, j - 1)
    else:
        memo[(i, j)] = 1 + min(D(i - 1, j),
                                D(i, j - 1),
                                D(i - 1, j - 1))
    return memo[(i, j)]
return D(n, m)
```

Cost

Memoization reduces the work to the $(n + 1)(m + 1)$ distinct prefix pairs.

Reading the Three Operations from the Table

Delete

Drop a_i :

$$D(i-1, j)$$

Insert

Insert b_j :

$$D(i, j-1)$$

Replace

Replace a_i by b_j :

$$D(i-1, j-1)$$

One extra cost

Each edit costs 1.

Edit Distance Complexity

Time

$$\Theta(nm)$$

Space

$$\Theta(nm)$$

for the full table.

Space optimization

If we only need the distance value, not the actual sequence of edits, two rows are enough.

Bottom-Up Levenshtein Algorithm

Pseudocode

```
for i = 0 to n:
    DP[i][0] = i
for j = 0 to m:
    DP[0][j] = j
for i = 1 to n:
    for j = 1 to m:
        if a[i] == b[j]:
            DP[i][j] = DP[i - 1][j - 1]
        else:
            DP[i][j] = 1 + min(DP[i - 1][j],
                               DP[i][j - 1],
                               DP[i - 1][j - 1])

return DP[n][m]
```

Cost

The table has one entry per prefix pair, so the running time is $\Theta(nm)$.

LCS vs Edit Distance

LCS

- maximize agreement
- recurrence uses max
- output is a common subsequence

Edit distance

- minimize transformation cost
- recurrence uses min
- output is a number or edit script

A Common DP Design Pattern

String and sequence DP often uses

- prefixes as states
- a 2D table
- a local decision based on the last symbols

Examples

LCS, edit distance, sequence alignment, dynamic time warping.

Memoization vs Tabulation vs Space Optimization

Memoization

Natural from the recurrence.

Tabulation

Clear evaluation order.

Space optimization

Store only what future states still need.

Rule of thumb

First get the state and recurrence right. Only then optimize the implementation.

DP Checklist for New Problems

- 1 What does one subproblem mean?
- 2 Which smaller subproblems does it depend on?
- 3 What are the base cases?
- 4 In what order can we fill the states?
- 5 Do we need only the value, or also the actual solution?

What to Remember

- DP turns exponential branching into polynomial work by reusing states.
- Fibonacci shows the core idea.
- Knapsack shows multidimensional state design.
- LCS and edit distance show how DP compares sequences.
- Good DP starts from a precise state definition.

Next Time

Coming next

We move from algorithm design to data structures for efficient lookup, updates, and large-scale approximate queries.

Algorithms for Data Science

Lecture #7: Dictionaries and Data Structures

Mateusz Buczynski

University of Warsaw

29.04.2026

Thanks to dr Krzysztof Fleszar for the base material.

Why Data Structures Matter

A data structure is more than storage

It stores data *and* maintains an invariant that makes some operations efficient.

Typical question

Which operations do we need to be fast: lookup, insertion, deletion, minimum, connectivity, approximate membership?

Dictionary: Abstract Data Type

Core object

A dictionary stores unique **key-value pairs**.

Main operations

- `Init()`
- `Check(k)`
- `Get(k)`
- `Add(k, v)`
- `Remove(k)`

Design goal

Choose a representation that makes the expensive operations cheap in the workload we actually care about.

A Running Theme: Trade-offs

Fast lookup

Often needs more structure.

Fast insertion

Often prefers looser structure.

Low memory

May hurt operation time.

Lecture viewpoint

We compare structures by operation cost, memory cost, and use case.

Direct Addressing

Assumption

Keys are integers in a known range:

$$k \in [0, M].$$

Representation

Store values in an array $a[0..M]$, where index = key.

Main idea

The key itself tells us where to look.

Direct Addressing: Costs

Operations

Init =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) = $\Theta(1)$

Add(k, v) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) = $\Theta(1)$

Add(k, v) = $\Theta(1)$

Space

$\Theta(M)$

When it is good

Excellent when the key range is small and dense.

Direct Addressing: Limitation

Sparse key spaces are expensive

If keys range up to one billion but only a thousand are present, direct addressing wastes memory badly.

Lesson

Perfect lookup time can be too expensive in space.

Unsorted Array of Key-Value Pairs

Representation

Store entries as a list of pairs:

$$[(k_1, v_1), (k_2, v_2), \dots, (k_n, v_n)]$$

Effect

Very simple, memory-efficient, but lookup requires scanning.

Unsorted Array: Operation Costs

Operations

Init =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(n)$

Get(k) =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(n)$

Get(k) = $\Theta(n)$

Add(k, v) =

Unsorted Array: Operation Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

in the general dictionary view, because we must first check whether k already exists.

If the New Key Is Guaranteed to Be New

Special case

If we know in advance that k is not yet present, then we can append (k, v) .

Cost with a dynamic array

$$\text{Add}(k, v) = \Theta(1) \text{ amortized}$$

Important distinction

Worst-case cost of one append can be large because resizing may be needed.

Dynamic Arrays and Resizing

Typical strategy

When the array is full, allocate a new array of double size and copy the old elements.

What happens

Most insertions are cheap; occasionally one insertion triggers a big copy.

Amortized Analysis: The Big Idea

Question

How expensive is one insertion *on average over a long sequence* of insertions?

Answer

Even though some insertions are expensive, the amortized cost stays

$$\Theta(1).$$

Why Amortized Cost Stays Constant

Doubling argument

Copies happen at sizes

$$1, 2, 4, 8, \dots$$

so the total number of copied elements after n insertions is

$$1 + 2 + 4 + \dots < 2n.$$

Conclusion

n insertions cost $O(n)$ total, so one insertion costs $O(1)$ amortized.

Sorted Array Dictionary

Invariant

Keep pairs sorted by key.

Main benefit

We can use binary search for $\text{Check}(k)$ and $\text{Get}(k)$.

Cost we pay

Keeping the order makes insertion harder.

Sorted Array: Costs

Operations

Init =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(\log n)$

Get(k) =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(\log n)$

Get(k) = $\Theta(\log n)$

Add(k, v) =

Sorted Array: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(\log n)$
Get(k)	=	$\Theta(\log n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

because elements may need to be shifted.

Sorted Linked List Dictionary

What it is

Store key-value pairs in a linked list sorted by key:

$$(k_1, v_1) \rightarrow (k_2, v_2) \rightarrow \cdots \rightarrow (k_n, v_n)$$

with $k_1 < k_2 < \cdots < k_n$.

How it works

Walk forward from the head until we either find key k or reach the first larger key.

Key difference from a sorted array

The order is maintained, but there is no binary search because we do not have random access.

Sorted Linked List: Array View

Logical order stored in array cells

The nodes can sit in arbitrary array positions; sorting is enforced by the next links.

Before insertion: head = 0, logical order $0 \rightarrow 2 \rightarrow 1 \rightarrow \text{nil}$

index	0	1	2
pair	(1, 10)	(4, 5)	(3, 9)
next	2	NIL	1

Add(2, 7): use free cell 3 and redirect one link

After insertion: head = 0, logical order $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{nil}$

index	0	1	2	3
pair	(1, 10)	(4, 5)	(3, 9)	(2, 7)
next	3	NIL	1	2

Sorted Linked List: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why these costs

We must scan nodes one by one to find the right position, even though the list is sorted.

Linked List Dictionary

What it is

Store key-value pairs in a plain linked list with no order requirement.

How it works

To search, scan node by node until key k is found or the list ends. To add a new pair, insert a new node at the front.

Main tradeoff

Insertion is simple, but lookups still require a full scan in the worst case.

Linked List: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

In the dictionary view, we still must first check whether k is already present before inserting a new node.

Exact Dictionary Structures So Far

Summary

Structure	Init	Check(k)	Get(k)	Add(k, v)	Space
Direct addressing	$\Theta(M)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(M)$
Unsorted array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Sorted array	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Sorted linked list	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Linked list	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Stack

Rule

Last In, First Out (LIFO).

Operations

- `push(x)`
- `top()`
- `pop()`

Typical uses

Recursion simulation, DFS, undo, parsing.

Queue

Rule

First In, First Out (FIFO).

Operations

- `enqueue(x)`
- `front()`
- `dequeue()`

Typical uses

BFS, simulations, scheduling, stream processing.

Why Stack and Queue Matter

Stack order

Go deep quickly, then come back.

Queue order

Process in arrival order, layer by layer.

Same data, different control flow

The chosen structure can completely change the algorithm behavior.

Priority Queue

Rule

Remove the element with best priority, not the oldest and not the newest.

Typical operations

- `insert(x)`
- `extractMin()` or `extractMax()`
- `decreaseKey()` in some implementations

Priority Queue via a Binary Heap

Heap invariant

In a min-heap, every node key is at most the keys of its children.

Costs

$\text{insert} = O(\log n), \quad \text{extractMin} = O(\log n)$

Why it matters

Dijkstra and many scheduling algorithms rely on priority queues.

Union-Find: Dynamic Connectivity

Goal

Maintain a partition of elements into disjoint sets.

Operations

- `Find(x)`: which set contains x ?
- `Union(x,y)`: merge the sets containing x and y

Typical use

Connectivity queries and Kruskal's MST algorithm.

Union-Find Representation

Forest view

Each set is represented by a rooted tree; the root is the representative.

Core rule

$\text{Find}(x)$ follows parent pointers to the root.

Why Union-Find Is Fast in Practice

Two key heuristics

- union by rank or size
- path compression

Practical result

Operations become almost constant-time for realistic input sizes.

Hash Table Dictionary

Main idea

Use a hash function $h(k)$ to map key k to one bucket in an array of size m .

Representation

Store an array of buckets and put pair (k, v) into bucket $h(k)$.

Why it helps

If keys spread evenly across buckets, each bucket stays short and operations become very fast.

Hash Functions

What a hash function does

A hash function maps a key to an array index:

$$h(k) \in \{0, 1, \dots, m - 1\}.$$

Simple example

For integer keys, one common idea is

$$h(k) = k \bmod m.$$

Example: if $m = 5$, then $h(12) = 2$ and $h(27) = 2$.

What we want

A good hash function spreads keys as evenly as possible, so buckets stay short.

Collisions and Chaining

Collision

Different keys can hash to the same bucket.

Standard fix: chaining

Each bucket stores a short linked list of all pairs whose keys hash there.

bucket 0 : $\emptyset$ bucket 1 : $(1, 10) \rightarrow (9, 4)$ bucket 2 : $(2, 7)$

Search rule

Hash first to find the bucket, then scan only inside that bucket.

Hash Table: Costs

Operations with chaining

Init = $\Theta(m)$
Check(k) = $\Theta(1)$ expected
Get(k) = $\Theta(1)$ expected
Add(k, v) = $\Theta(1)$ expected

Important caveat

Worst case is $\Theta(n)$ if many keys fall into the same bucket.

Assumption behind the fast case

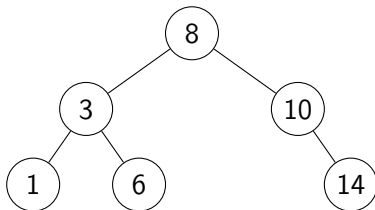
A good hash function and a bounded load factor.

Binary Search Tree (BST)

Invariant

For each node with key k :

- all keys in the left subtree are smaller than k
- all keys in the right subtree are larger than k



BST Search Logic

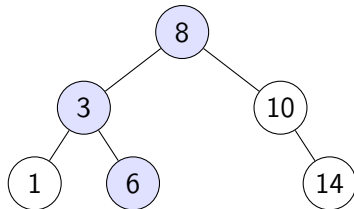
Decision rule

At a node with key k :

- if target $< k$, go left
- if target $> k$, go right
- if target $= k$, stop

Cost

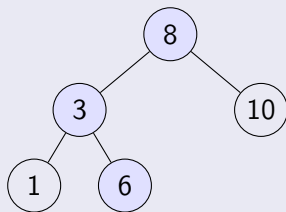
Proportional to the tree height h .



search for key 6: $8 \rightarrow 3 \rightarrow 6$

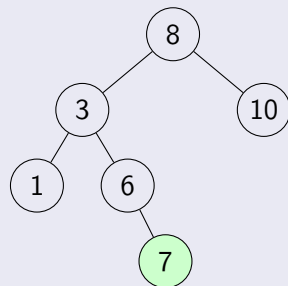
BST Insertion Example

Before adding 7



path for 7: $8 \rightarrow 3 \rightarrow 6$

After adding 7

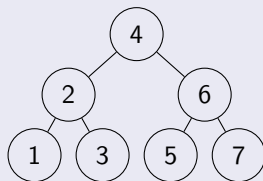


Insertion rule

Follow the BST comparisons until you reach an empty child pointer, then insert there.

BSTs Can Be Excellent or Terrible

Balanced shape



$$h = \Theta(\log n)$$

Degenerate shape



$$h = \Theta(n)$$

Lesson

The same keys can produce very different performance depending on the tree shape.

How a BST Becomes Unbalanced

Insert sorted keys

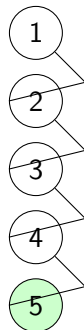
If we insert

1, 2, 3, 4, 5

in this order, each new key becomes the right child of the previous one.

Consequence

The BST behaves like a linked list, so lookup and insertion degrade to linear time.



Self-Balancing BSTs

Idea

Maintain extra structural rules so the height stays logarithmic.

Examples

AVL trees, red-black trees.

Why rebalancing matters

Without it, BST guarantees collapse on unlucky insertion orders.

AVL Trees

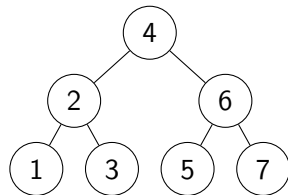
AVL invariant

At every node, the heights of the left and right subtrees differ by at most 1.

Effect

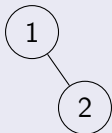
$$h = \Theta(\log n)$$

so search, insertion, and deletion are logarithmic.



AVL Insertion and Rebalancing

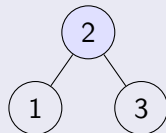
Before add(3)



After add(3)



After left rotation

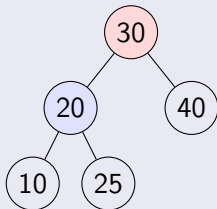


What happened

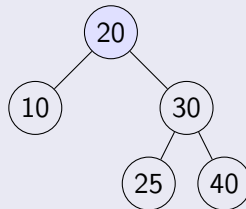
The new key made node 1 too right-heavy, so one local rotation restored balance.

Rotations: Local Repairs

Before right rotation



After right rotation



Key point

Rotations are local changes that preserve the BST order invariant while reducing height.

Exact Structures: Intermediate Summary

What improves from left to right?

array scan $\rightarrow$ binary search / hashing $\rightarrow$ balanced tree

What the structures buy us

More invariants or better key distribution usually mean faster queries.

Why Probabilistic Data Structures?

Big-data motivation

Sometimes exact answers are too expensive in time or memory.

Alternative

Accept a small, controlled probability of error to save massive space.

Bloom Filter: Goal

Task

Support approximate membership queries:

“Have we seen key x before?”

Promise

Very small memory footprint, very fast queries.

Bloom Filter: Structure

Representation

- a bit array of length m
- k hash functions

Insert x

Hash x with all k functions and set the corresponding bits to 1.

Bloom Filter: Query Logic

Check x

If at least one of the k bits is 0, then x is definitely absent.

But if all bits are 1

x is *probably* present.

Error type

False positives are possible; false negatives are not.

When Bloom Filters Are Useful

- web crawling: have we already visited this URL?
- databases: avoid expensive disk lookups for definitely absent keys
- networking and caches

Trade-off

We do not store values, only approximate set membership.

HyperLogLog: Goal

Task

Estimate the number of distinct elements in a huge stream.

Why exact counting is hard

An exact set of all distinct values may be too large to store.

HyperLogLog: Core Intuition

Hash values look random

Rare patterns, such as many leading zeros, are evidence that we have seen many distinct elements.

Main idea

Split the stream into buckets, track a statistic related to the longest run of leading zeros, then aggregate.

Why HyperLogLog Is Popular

Pros

- tiny memory
- mergeable summaries
- good accuracy for large streams

Cons

- approximate only
- not useful when exact identities are needed

Bloom Filters vs HyperLogLog

Bloom filter

Approximate membership.

HyperLogLog

Approximate distinct count.

Big idea

Probabilistic structures answer a smaller question than a full dictionary, but do it at massive scale.

Similarity Search at Scale

Problem

Given a query item, find stored items that are similar to it.

Applications

Near-duplicate documents, recommendation, image retrieval, embedding search.

Challenge

Exact nearest-neighbor search can be too slow in high dimensions.

Locality Sensitive Hashing (LSH)

Main principle

Use hash functions designed so that similar items are more likely to collide than dissimilar items.

Interpretation

Hashing is no longer about exact equality; it becomes a tool for approximate similarity search.

Why LSH Helps

Workflow

- 1 hash all items into buckets
- 2 hash the query
- 3 compare mainly with items in the same or nearby buckets

Effect

Much less brute-force scanning.

Where LSH Fits in the Big Picture

Exact dictionary mindset

Same key means same object.

LSH mindset

Similar objects should land close together.

Trade-off

We gain speed by accepting approximate candidate generation.

Choosing the Right Structure

Ask first

What is the real question?

- exact lookup by key
- ordered lookup
- repeated min extraction
- dynamic connectivity
- approximate membership
- approximate distinct counting
- approximate similarity search

Operational Summary

Need	Typical structure
FIFO processing	queue
LIFO processing	stack
best-priority extraction	priority queue / heap
dynamic connectivity	union-find
exact lookup by key	hash table
exact ordered map	balanced BST
approximate membership	Bloom filter
approximate cardinality	HyperLogLog
similarity candidate search	LSH

What to Remember

- Data structures are about invariants plus operation costs.
- Dynamic arrays need amortized analysis.
- Hash tables give expected constant-time exact lookup, but not sorted order.
- Trees trade extra structure for faster lookup.
- Support structures like queues and heaps shape algorithm behavior.
- Probabilistic structures are essential at modern scale.

Next Time

Coming next

We will use many of these structures inside graph algorithms, network analysis, and spatial search.

Algorithms for Data Science

Lecture #8: Graphs, Networks, and Spatial Search

Mateusz Buczynski

University of Warsaw

13.05.2026

Thanks to dr Krzysztof Fleszar for the base material.

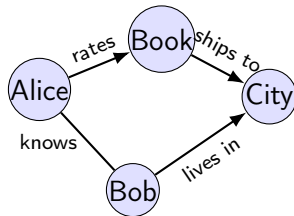
Why Graphs Matter

Graphs model relationships

Vertices represent objects, and edges represent interactions, transitions, or dependencies.

Examples

Road networks, social networks, recommendation systems, molecules, web links, game states.



Basic Vocabulary

Notation

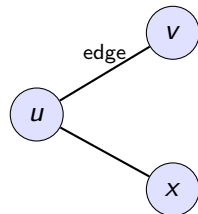
$$G = (V, E)$$

or, for weighted graphs,

$$G = (V, E, w).$$

Edge

A connection between two vertices.



a graph is a set of vertices and edges

Vertex

A node in the graph.

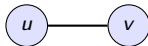
Parallel

Graphs can have multiple parallel connections.

Directed, Undirected, Weighted

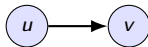
Undirected

Friendship, symmetric relation.



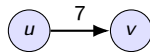
Directed

Following, one-way streets.



Weighted

Travel time, cost, distance.



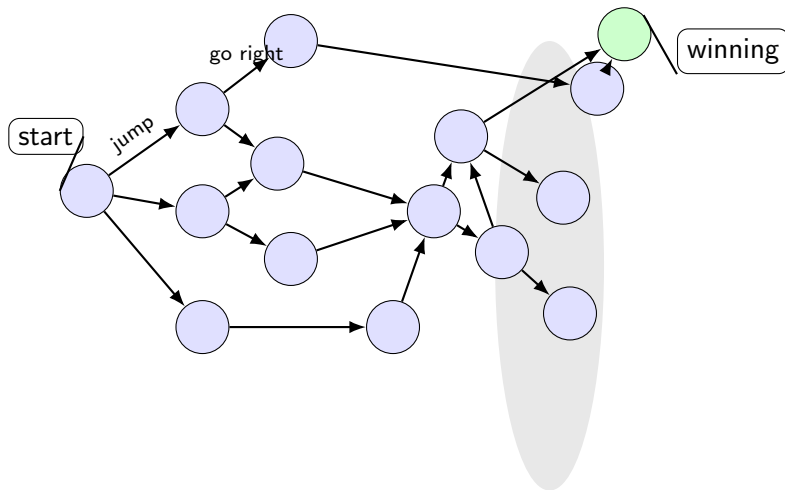
Modeling matters

The right graph model is already half of the algorithmic solution.

Example: Video Game

state = (position, time, gold)

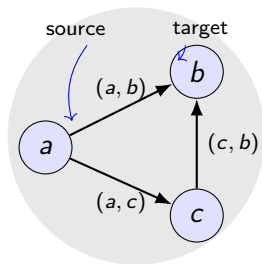
vertices = states



How to Represent a Graph?

Conventions

n = number of vertices, m = number of edges



Same graph, different data structures

We can store the same edges as:

- an adjacency matrix
- adjacency lists
- an edge list

	a	b	c
a	0	1	1
b	0	0	0
c	0	1	0

$a : b \rightarrow c \rightarrow \text{None}$

Adjacency Matrix vs Adjacency Lists

Adjacency matrix

$\Theta(n^2)$ space

Fast edge-existence check.

	<i>a</i>	<i>b</i>	<i>c</i>
<i>a</i>	0	1	1
<i>b</i>	1	0	0
<i>c</i>	1	0	0

Adjacency lists

$\Theta(n + m)$ space

Usually better for sparse graphs.

a : *b*, *c*
b : *a*
c : *a*

Default choice

For algorithmic work on sparse graphs, adjacency lists are usually the right representation.

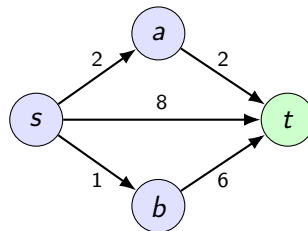
Shortest vs Cheapest Paths

Shortest path

Minimize the number of edges.

Cheapest path

Minimize total edge weight.



fewest edges: $s \rightarrow t$ (cost 8)

cheapest: $s \rightarrow a \rightarrow t$ (cost 4)

Key warning

These are the same only when all edges have equal weight.

What Could the Edge Mean?

Scenario

You want to build a route-planning tool for a delivery company.

What Could the Edge Mean?

Scenario

You want to build a route-planning tool for a delivery company.

What should an edge weight represent?

- physical distance
- travel time
- fuel cost
- tolls
- expected delay under traffic

Breadth-First Search (BFS): Goal

Input

An unweighted graph and a start vertex s .

Output

For every reachable vertex v , compute the length of the shortest path from s to v in number of edges.

Why BFS Uses a Queue

Core idea

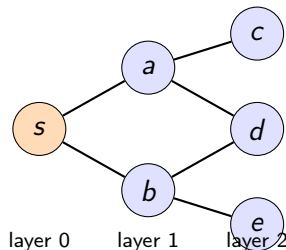
Vertices must be processed in order of discovery time.

Queue effect

First discovered, first expanded. This creates layers:

$0, 1, 2, 3, \dots$

from the source.



BFS Algorithm Template

Data

Queue Q , distance array $dist$, optional parent array $parent$.

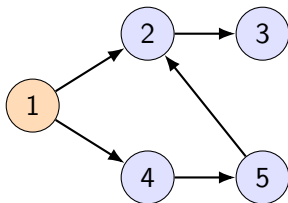
Flow

- 1 initialize all distances to $+\infty$
- 2 set $dist[s] = 0$, push s into Q
- 3 while Q is non-empty, pop the front vertex
- 4 relax all still-unvisited neighbors

Breadth-First Search (BFS): Visual View

Goal

For each vertex w , compute the shortest s - w path in number of edges.



vertex	1	2	3	4	5
dist	0	1	2	1	2

Queue snapshot during the run

$[1] \rightarrow [2, 4] \rightarrow [4, 3] \rightarrow [3, 5] \rightarrow [5]$

Algorithm sketch

```

dist[s] = 0
enqueue(s)
while Q not empty do
  v = extractOldest()
  for each edge (v,w) do
    if dist[w] not set then
      dist[w] = dist[v] + 1
    enqueue(w)
  
```

FIFO queue $\Rightarrow$ process vertices layer by layer.

BFS Complexity

With adjacency lists

Each vertex is enqueued at most once and each edge is scanned at most once.

Running time

$$\Theta(n + m)$$

Use it when

The graph is unweighted or all edge costs are identical.

Depth-First Search (DFS): Goal

Different behavior, different objective

DFS explores one branch deeply before backtracking.

Typical uses

Connectivity, topological ideas, cycle detection, component discovery, search-space exploration.

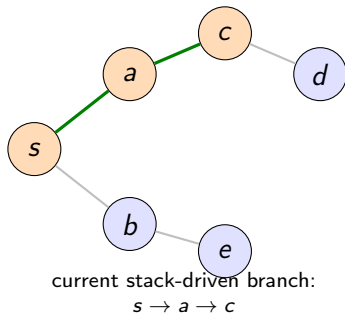
Why DFS Uses a Stack

Core idea

The most recently discovered unexplored vertex should be processed next.

Stack effect

Go deep first, then come back.



DFS Algorithm Template

Iterative version

Keep an explicit stack of vertices to process.

Flow

- 1 push s
- 2 pop a vertex
- 3 if not visited, mark it and push its neighbors

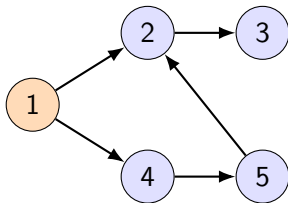
Equivalent recursive view

The call stack plays the role of the explicit stack.

Depth-First Search (DFS): Visual View

Goal

Explore one branch as deeply as possible before backtracking.



vertex	1	2	3	4	5
visit order	1	2	3	4	5

Stack snapshot during the run

$[1] \rightarrow [4, 2] \rightarrow [4, 3] \rightarrow [4] \rightarrow [5]$

Algorithm sketch

```

push(s)
while S not empty do
  v = pop()
  if v not visited then
    mark v visited
    for each edge (v,w) do
      push(w)
  
```

LIFO stack $\Rightarrow$ keep following the newest branch.

DFS Complexity

With adjacency lists

Like BFS, DFS visits every vertex and scans every edge at most once.

Running time

$$\Theta(n + m)$$

BFS vs DFS

BFS

- queue
- layer by layer
- shortest paths in unweighted graphs

DFS

- stack or recursion
- go deep first
- better for exploration and structure

Worked Example: BFS by Hand

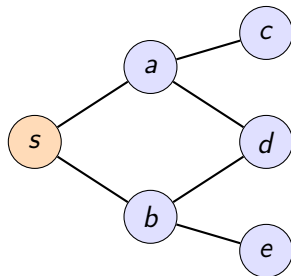
Task

Start at vertex s . In what order does BFS discover the vertices?

Assume alphabetical neighbor order

Track three things:

- queue contents
- discovery order
- distance labels



Instructor use

Good point to pause for 3–5 minutes and have students simulate the queue.

Worked Example: DFS by Hand

Task

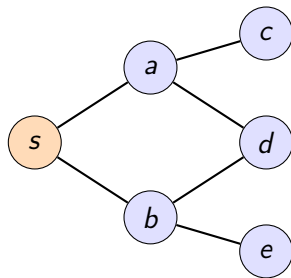
Run DFS from s on the same graph.

Ask

How does the order change if neighbors are processed alphabetically?

Compare with BFS

Same graph, same source, very different exploration pattern.



Checkpoint: BFS or DFS?

Question 1

You need shortest paths in an unweighted graph. Which traversal is the right default?

Question 2

You need to explore a search tree deeply and backtrack naturally. Which traversal fits better?

Question 3

How much can the output order depend on the order of neighbors in the adjacency lists?

Dijkstra's Algorithm: Problem Setup

Input

A weighted graph with nonnegative edge weights and a source s .

Goal

Compute the cheapest path from s to every vertex.

From BFS to Dijkstra

What changes?

In BFS, the next vertex to expand is the oldest in the queue.

In Dijkstra

The next vertex should be the currently cheapest not-yet-finalized vertex.

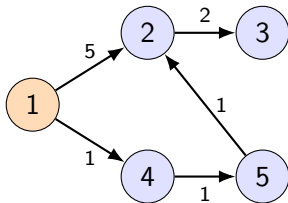
Data structure shift

Queue $\rightarrow$ priority queue.

Dijkstra's Algorithm: Visual View

Goal

For each vertex w , compute the cheapest s - w path.



vertex	1	2	3	4	5
dist	0	3	5	1	2

Priority queue snapshot during the run

$(1, 0) \rightarrow [(4, 1), (2, 5)] \rightarrow [(5, 2), (2, 5)]$
 $\rightarrow [(2, 3)] \rightarrow [(3, 5)]$

Algorithm sketch

```

dist[s] = 0
add(s, 0)
while PQ not empty do
  v = extractMin()
  for each edge (v, w) do
    if dist[w] > dist[v] + c(v, w) then
      dist[w] = dist[v] + c(v, w); add(w, dist[w])
  
```

Priority queue $\Rightarrow$ always expand the cheapest candidate next.

Dijkstra: Relaxation Rule

At an edge (u, v) of cost $w(u, v)$

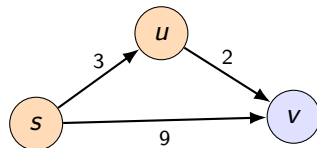
If going through u improves the best known cost to v , update it:

$$\text{dist}[v] > \text{dist}[u] + w(u, v)$$

$$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$$

Optional extra data

Store $\text{parent}[v] = u$ to reconstruct the actual path.



$\text{dist}[u] = 3$
old $\text{dist}[v] = 9$
new via u : $3 + 2 = 5$

Why Dijkstra Works

Key invariant

When a vertex is extracted from the priority queue as the cheapest candidate, its distance is already final.

Why nonnegative weights matter

No later path can sneak in and become cheaper through a negative edge.

Dijkstra Complexity

Adjacency lists + binary heap

$$O((n + m) \log n)$$

Interpretation

Good for sparse weighted graphs.

Recovering Actual Paths

Parent array idea

Whenever we improve $dist[v]$, set

$$parent[v] = u.$$

To reconstruct path to t

Follow

$$t, parent[t], parent[parent[t]], \dots$$

back to the source, then reverse the sequence.

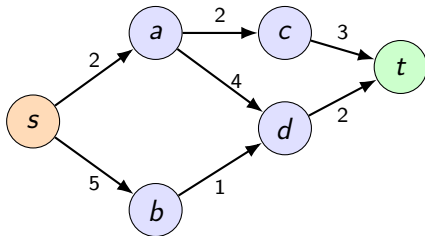
Worked Example: Dijkstra Trace

Task

Start from s . Which vertex is finalized next after each extraction?

Track

- current priority queue minimum
- tentative distance table
- parent pointers



Single-Source vs All-Pairs

Single-source

One source s , distances to all vertices.

All-pairs

Distances between every ordered pair (u, v) .

Typical question

If many source-target pairs will be queried, all-pairs preprocessing can be worth it.

Floyd-Warshall: Goal

Input

A weighted directed graph.

Output

An $n \times n$ matrix of shortest-path distances between all vertex pairs.

Floyd-Warshall: DP State

State meaning

After stage k ,

$$\delta[v][w]$$

is the cheapest cost from v to w using only internal vertices from

$$\{1, \dots, k\}.$$

Floyd-Warshall: Initialization

Base case $k = 0$

$$\delta[v][w] = \begin{cases} 0, & v = w, \\ w(v, w), & \text{if edge } (v, w) \text{ exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

Interpretation

With no allowed internal vertices, only direct edges and trivial paths remain.

Floyd-Warshall: Update Rule

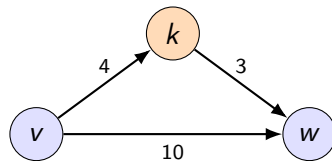
At stage k

For every pair (v, w) , compare

$$\delta[v][w]$$

with the path that goes through k :

$$\delta[v][k] + \delta[k][w].$$



direct: 10
through k : $4 + 3 = 7$

Transition

$$\delta[v][w] \leftarrow \min(\delta[v][w], \delta[v][k] + \delta[k][w]).$$

Floyd-Warshall: Why the DP Works

Decision at stage k

The best path from v to w :

- either does not use vertex k internally
- or does use k internally

This is classical DP

The recurrence splits the answer into two smaller, already solved possibilities.

Floyd-Warshall Complexity

Time

$$\Theta(n^3)$$

Space

$$\Theta(n^2)$$

Use it when

The graph is dense, the vertex set is not too large, or all-pairs distances are needed.

Worked Example: One Floyd-Warshall Update

Mini-instance

Suppose we currently know

$$\delta[a][d] = 12, \quad \delta[a][c] = 5, \quad \delta[c][d] = 4.$$

At stage $k = c$, what should happen to $\delta[a][d]$?

Discussion

Compare the old direct best value 12 with the route through c :

$$5 + 4 = 9.$$

So the entry should be improved to 9.

Why this helps pedagogically

Students often understand Floyd-Warshall better from one table update than from the

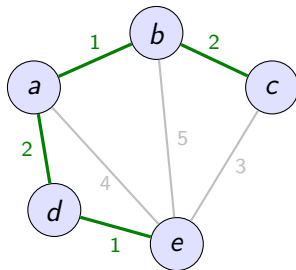
Minimum Spanning Tree (MST)

Input

A connected, weighted, undirected graph.

Goal

Choose a subset of edges that connects all vertices with minimum total weight and no cycles.



Kruskal's Algorithm

Greedy rule

Sort edges by weight from smallest to largest, then add an edge if it does not create a cycle.

Cycle test tool

Use union-find to check whether the endpoints are already in the same component.

Why Kruskal Works

Intuition

Whenever we connect two currently separate components with the cheapest possible edge, we never regret that choice.

Key property

The algorithm grows a forest that gradually merges into one spanning tree.

Kruskal Complexity

Dominant cost

Sorting the edges:

$$O(m \log m)$$

With union-find

Connectivity checks are nearly constant-time in practice.

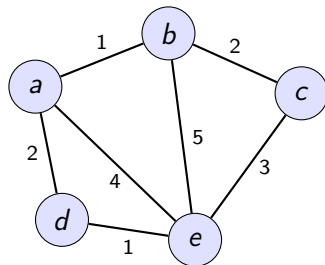
Worked Example: Kruskal by Hand

Task

Process edges in increasing order of weight and decide which ones enter the MST.

Class prompt

At which step is an edge rejected because it would create a cycle?



Break Point

Part 1 and Part 2 complete

Suggested break: 10–15 minutes

Before the break

Students should now be able to choose between BFS, DFS, Dijkstra, Floyd-Warshall, and Kruskal for standard path and connectivity tasks.

Graphs as Networks

Beyond paths

In data science, graphs often represent influence, importance, traffic, or diffusion.

Typical question

Which nodes are central, influential, or structurally important?

PageRank: Core Intuition

Web interpretation

A page is important if important pages point to it.

Recursive scoring idea

Importance flows along incoming links.

This is a fixed-point problem

Scores depend on other scores.

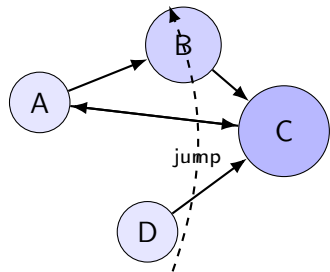
PageRank as Random Surfer Model

Mental picture

A surfer repeatedly follows outgoing links, with occasional random jumps.

Interpretation

The stationary probability of being at a page becomes its rank score.



Centrality Measures

Big question

What does “important node” mean?

Answer

There is no single notion of importance; different centrality measures capture different roles.

Degree Centrality

Definition

Count the number of incident edges.

Interpretation

Nodes with many direct connections are locally influential or highly active.

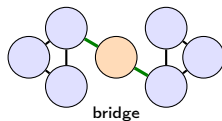
Closeness and Betweenness Centrality

Closeness

Nodes with short average distance to others are globally well positioned.

Betweenness

Nodes that lie on many shortest paths act as bridges or bottlenecks.



Discussion Exercise: Which Node Is Most Important?

Prompt

In a transportation network, which node should be considered the most important?

Possible answers depend on the metric

- highest degree: many direct links
- highest closeness: fast access to the whole system
- highest betweenness: critical transfer hub
- highest PageRank: receives traffic from already important places

Teaching goal

Students should stop thinking of “importance” as a single objective fact.

A* Search: Why Another Path Algorithm?

Motivation

In pathfinding, we often care about one source-target pair, not all distances from the source.

Goal

Use extra problem knowledge to search more directly toward the target.

A* Search: Scoring Rule

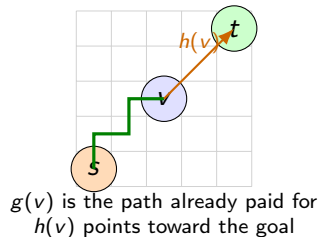
Priority key

For a candidate vertex v , use

$$f(v) = g(v) + h(v)$$

where

- $g(v)$: cost from source to v
- $h(v)$: heuristic estimate from v to target



A* Search: Connection to Dijkstra

Special case

If

$$h(v) = 0 \quad \text{for all } v,$$

then A* becomes Dijkstra.

Interpretation

A* is Dijkstra plus guidance.

When A* Works Well

Good heuristic

The heuristic should be informative but not overoptimistic in the wrong direction.

Typical example

Euclidean distance to the goal in map routing.

Use case

Games, navigation, robotics, route planning.

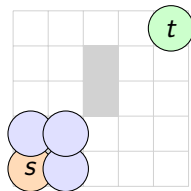
Worked Example: A* on a Grid

Task

Suppose movement cost is 1 per step and $h(v)$ is Manhattan distance to the goal.

Ask

Which frontier node has the smallest $f = g + h$ score after the first few expansions?



The Nearest Neighbor Problem

Task

Given a query point, find the closest stored point under a distance measure.

Applications

Recommendation, image retrieval, anomaly detection, classification, vector search.

k-NN Classification

Basic rule

Find the k closest training points and let them vote on the label.

Algorithmic challenge

Training is cheap, but prediction can be expensive if we scan all stored points.

Why Brute Force Becomes Expensive

If there are N points in d dimensions

One exact query by full scan typically costs

$$\Theta(Nd).$$

Problem

This can be too slow when N is very large or queries are frequent.

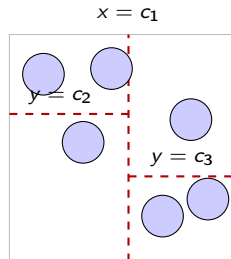
k-d Trees

Core idea

Recursively split space by axis-aligned cuts.

Node meaning

Each internal node stores one coordinate split and partitions the point set into two regions.



How k-d Tree Search Prunes Work

Main rule

First follow the branch that contains the query point.

Then

Only explore the other branch if its region could still contain a closer point than the best one found so far.

When k-d Trees Work Well

Best setting

Low or moderate dimension, reasonably structured data.

Warning

In high dimensions, pruning often weakens and brute force may become competitive again.

Ball Trees

Alternative partition idea

Instead of axis-aligned boxes, organize points into nested metric balls.

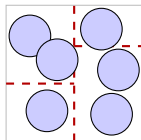
Why useful

Ball trees can adapt better than k-d trees when Euclidean coordinates are not the whole story.

k-d Trees vs Ball Trees

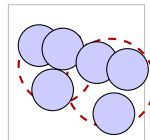
k-d tree

Axis-aligned partition.



Ball tree

Metric-space partition by radius.



Practical note

Both are exact search structures, but their usefulness depends on geometry and dimension.

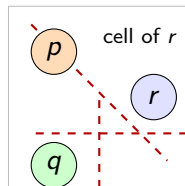
Voronoi Diagrams

Geometric view of nearest neighbors

Partition the plane into regions where each region contains all points closest to one site.

Interpretation

If the query lies inside the Voronoi cell of site p , then p is the nearest neighbor.



Spatial Search Exercise

Prompt

You have a million 2D points and many nearest-neighbor queries.

Discuss in pairs

Which method would you try first?

- brute-force scan
- k-d tree
- ball tree

What changes if the points live in 200 dimensions instead?

Target insight

The geometry of the data can matter as much as the asymptotic complexity written on the slide.

Which Graph or Spatial Algorithm Should I Use?

Problem	Typical tool
unweighted shortest paths	BFS
graph exploration / components	DFS
weighted shortest paths	Dijkstra
all-pairs shortest paths	Floyd-Warshall
minimum total network cost	Kruskal / MST
one target with heuristic guidance	A*
nearest-neighbor search	k-d tree / ball tree / scan

Final Checkpoint Questions

- 1 Why is BFS correct for shortest paths only in unweighted graphs?
- 2 Why does Dijkstra require nonnegative edge weights?
- 3 When is Floyd-Warshall a better fit than repeated single-source search?
- 4 Why is an MST not the same thing as a shortest-path tree?
- 5 When do k-d trees start to lose their advantage?

The Bigger Picture

- Graph algorithms solve path, flow, and connectivity questions.
- Network analysis asks which vertices matter most.
- Spatial search asks how to organize geometry for fast queries.
- The right model and the right data structure are as important as the algorithm itself.

What to Remember

- BFS and DFS are the two foundational graph traversals.
- Dijkstra and Floyd-Warshall solve different shortest-path settings.
- Kruskal connects a network cheaply, not quickly.
- PageRank and centrality scores turn graphs into analytical tools.
- A* and spatial trees use structure to cut search time.

Next Time

Coming next

We move from efficient algorithms to the boundary of efficient computation: lower bounds, reductions, NP-completeness, and approximation.

Algorithms for Data Science

Lecture #9: Limits of Efficient Computation

Mateusz Buczynski

University of Warsaw

27.05.2026

Thanks to dr Krzysztof Fleszar for the base material.

A Core Algorithmic Question

Given an algorithm, ask

Can we do better?

Two possible outcomes

- yes: there is room for asymptotic improvement
- no: a lower bound blocks that improvement

Lower Bounds Tell Us What Is Impossible

Meaning

A lower bound of $\Omega(f(n))$ means that every algorithm needs at least that much time in the chosen model.

Decision rule

If our algorithm already matches the lower bound asymptotically, then it is optimal in that model.

Sorting as the Familiar Example

Comparison model

Every comparison-based sorting algorithm needs

$$\Omega(n \log n)$$

comparisons in the worst case.

Consequence

No comparison sort can be worst-case $O(n)$.

Decision Problems

Format

The answer must be only

YES or NO.

Examples

- Is there a path from s to t ?
- Is there a knapsack solution of value at least K ?
- Is a Boolean formula satisfiable?

Example: Is There a Path from s to t ?

Input

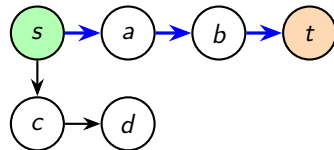
A directed graph G , a source s , and a target t .

Question

Is there a directed path from s to t ?

Answer

YES or NO — nothing else.



Path $s \rightarrow a \rightarrow b \rightarrow t$ exists: YES.

Formal View: Is $w \in S$?

Encoding

Every instance (graph, numbers, formula, ...) is encoded as a string w over some alphabet.

Decision problem as a language

A decision problem is identified with a **language**

$$S \subseteq \Sigma^*$$

the set of all encodings of YES-instances.

The canonical form

Given input w , decide: $w \in S$?

Path problem

$$S = \{ \langle G, s, t \rangle \mid G \text{ has a path from } s \text{ to } t \}$$

Why This Formulation?

- **Why yes/no problems?**
 - simpler to describe theoretically
 - decision version is at least as hard as the optimization version
- **Why encode input as a word?**
 - gives a precise, model-independent definition
 - input size = number of letters, so complexity is well-defined

Time Hierarchy

Time Hierarchy Theorem

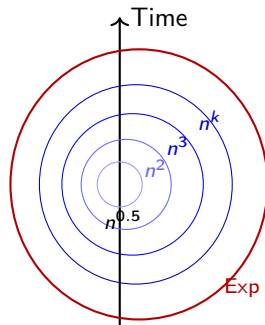
Let $f(n)$, $g(n)$ be running time functions.
 If g grows faster than f (significantly),
 then there is a problem solvable
 in $O(g(n))$ but not in $O(f(n))$.

Corollary 1

For every k , there is a problem solvable
 in polynomial time but not in $O(n^k)$.

Corollary 2

There is a problem solvable in exponential time
 but not in polynomial time.



Class P

Definition

P is the class of decision problems solvable in polynomial time.

Interpretation

These are the problems we usually call efficiently solvable.

Examples

Sorting, shortest paths, MST, bipartite matching, linear programming.

Polynomial in What?

Important detail

Running time must be polynomial in the **input length**, not just in a numeric parameter that appears inside the input.

Why this matters

The number $M = 10^9$ is huge, but its binary encoding length is only about 30 bits.

Knapsack and Pseudo-Polynomial Time

Reminder from DP

The standard DP for knapsack runs in

$$\Theta(nM).$$

But

This is polynomial in the numeric capacity M , not in the bit-length of M .

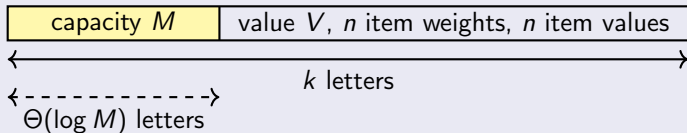
Name

Such running time is called **pseudo-polynomial**.

Can one pack items into the knapsack of total value at least V ?

Input: capacity M , value V , n item weights, n item values
highlighted

Input encoding has k letters total



Key fact

To write any integer M , $\Theta(\log M)$ letters are needed and suffice.
(e.g. $M = 1000$ uses $4 = 1 + \log_{10} 1000 = \Theta(\log M)$ digits)

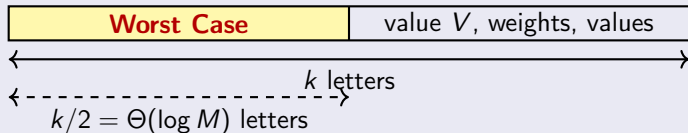
Knapsack: No, It Is Exponential

KNAPSACK	believed not to be in P
----------	---------------------------

Can one pack items into the knapsack of total value at least V ?

Input: capacity M , value V , n item weights, n item values
highlighted

Worst-case input: M dominates



$$\log M = \Theta(k) \implies M = 2^{\Theta(k)}$$

$$\Theta(n \cdot M) = 2^{\Omega(k)} \Rightarrow \text{No, exponential!}$$

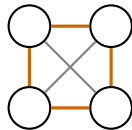
Hamiltonian Cycle: Believed Not in P

HAMILTONIAN CYCLE

Is there a cycle visiting each vertex exactly once?

Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known



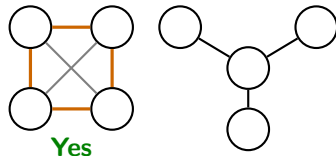
Hamiltonian Cycle: Believed Not in P

HAMILTONIAN CYCLE

Is there a cycle visiting each vertex exactly once?

Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known



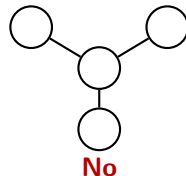
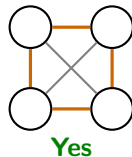
Hamiltonian Cycle: Believed Not in P

HAMILTONIAN CYCLE

Is there a cycle visiting each vertex exactly once?

Status

- NP-complete
- best known exact algorithms: $O(2^n \cdot n^2)$
- no polynomial-time algorithm is known



Traveling Salesman: Believed Not in P

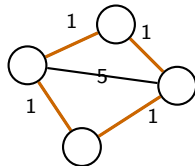
TRAVELING SALESMAN (DECISION)

Is there a tour visiting all vertices of total cost $\leq C$?

(vertices may be revisited; must return to start)

Status

- NP-complete
- no polynomial-time algorithm is known
- approximable within $3/2$ in metric case (Christofides)



Traveling Salesman: Believed Not in P

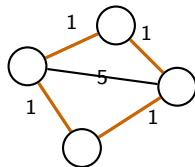
TRAVELING SALESMAN (DECISION)

Is there a tour visiting all vertices of total cost $\leq C$?

(vertices may be revisited; must return to start)

Status

- NP-complete
- no polynomial-time algorithm is known
- approximable within $3/2$ in metric case (Christofides)



Tour cost: $1+1+1+1 = 4$. $C = 5$? **Yes.**

SAT: Believed Not in P

BOOLEAN SATISFIABILITY (SAT)

Is there a true/false assignment making the formula true?

Example formula

$$(a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg b)$$

SAT: Believed Not in P

BOOLEAN SATISFIABILITY (SAT)

Is there a true/false assignment making the formula true?

Example formula

$$(a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg b)$$

Assignment $a = \top$, $b = \perp$: $(\top \vee \perp) \wedge (\perp \vee \top) \wedge (\top \vee \top) = \top$. **Yes.**

Status

- first proven NP-complete (Cook–Levin, 1971)
- best known exact algorithms: $O(2^n)$ in the number of variables
- no polynomial-time algorithm is known

Polynomial-Time Reductions

Main idea

To show that problem A is no harder than problem B , transform instances of A into instances of B .

Notation

$$A \leq_p B$$

means A reduces to B in polynomial time.

Why Reductions Are Powerful

If

$$A \leq_p B$$

and B has a polynomial-time algorithm,

then

A also has a polynomial-time algorithm.

Interpretation

An algorithm for B can be reused as a black box for A .

Reduction Direction Matters

Warning

$$A \leq_p B$$

does *not* mean B reduces to A .

How to read it

If B is easy, then A is easy. If A is hard, then B must be at least that hard.

Example Reduction: Ham. Cycle $\leq_p$ TSP

Construction

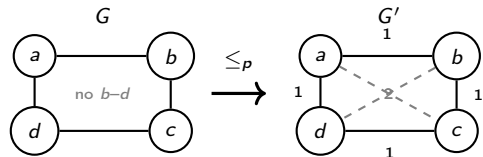
Given graph $G = (V, E)$, build a complete weighted graph G' :

$$w(u, v) = \begin{cases} 1 & \{u, v\} \in E \\ 2 & \{u, v\} \notin E \end{cases}$$

Ask TSP: is there a tour in G' of cost $\leq |V|$?

Correctness

G has a Ham. cycle $\iff G'$ has a tour of cost $\leq |V|$.



Tour $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ costs $4 = |V|$. **Yes.**

Consequence

If TSP $\in P$ then Ham. Cycle $\in P$.

A Canonical Example Family

Classic hard decision problems

- SAT
- Hamiltonian Cycle
- Traveling Salesman (decision)
- KNAPSACK (decision)

The big surprise

These problems are connected by reductions; they are not isolated accidents.

Class NP

Meaning

NP stands for *nondeterministic polynomial time*.

Practical intuition

A YES answer has a certificate that can be checked in polynomial time.

What Does “Easy to Verify” Mean?

SAT

Certificate: an assignment of truth values.

Knapsack

Certificate: a proposed subset of items.

Common theme

Checking a proposed solution is much easier than searching for one.

Containment

Basic fact

$$P \subseteq NP$$

Why

If we can solve a problem quickly, then we can certainly verify a proposed answer quickly.

NP-Hard and NP-Complete

NP-hard

At least as hard as every problem in NP .

NP-complete

In NP and NP-hard.

Interpretation

NP-complete problems are the hard core inside NP .

Why NP-Complete Problems Matter So Much

Key implication

If one NP-complete problem has a polynomial-time algorithm, then

$$P = NP.$$

Therefore

A breakthrough on one canonical NP-complete problem would propagate to all problems in NP .

The P vs NP Question

One of the biggest open problems in mathematics and computer science

$$P \stackrel{?}{=} NP$$

Two worlds

- if $P = NP$: efficient exact algorithms exist for all NP problems
- if $P \neq NP$: some efficiently verifiable problems are inherently hard to solve exactly

How to Think in Practice

For NP-complete problems

Do not expect a general exact polynomial-time algorithm unless a major theoretical breakthrough happens.

So what do we do?

Use structure, special cases, approximation, heuristics, randomization, or parameterized methods.

Video

<https://www.youtube.com/watch?v=x36UmiSiEzc>

Approximation Algorithms

Idea

Return a solution that may be non-optimal, but comes with a provable quality guarantee.

Why useful

When exact optimization is too slow, a guaranteed near-optimal solution can still be very valuable.

Approximation vs Heuristics

Approximation

Has a formal bound on solution quality.

Heuristic

Often fast and useful, but no provable guarantee is required.

Metric TSP Approximation: Setup

Optimization problem

Visit all cities and return to the start with minimum total cost.

Assumption for approximation

We work in the metric case, where the triangle inequality holds.

Why the MST Helps for TSP

Key inequality

If T is a minimum spanning tree and OPT is the optimal TSP tour, then

$$\text{cost}(T) \leq \text{cost}(OPT).$$

Reason

Removing one edge from any TSP tour leaves a spanning tree.

A 2-Approximation for Metric TSP

Algorithm

- 1 compute an MST
- 2 double its edges to get a closed walk
- 3 shortcut repeated vertices

Guarantee

$$\text{cost}(\text{tour}) \leq 2 \cdot \text{cost}(\text{OPT}).$$

Why the 2-Approximation Bound Works

Step 1

$$\text{cost}(T) \leq \text{cost}(OPT)$$

Step 2

Doubling tree edges gives cost

$$2 \cdot \text{cost}(T).$$

Step 3

Shortcutting does not increase cost in metric spaces.

What Approximation Gives Us

- a principled response to NP-hard optimization
- predictable quality
- scalable practical algorithms in many settings

Data-science relevance

Large optimization problems often need good answers fast, not perfect answers slowly.

What to Remember

- Lower bounds explain when improvement is impossible.
- P means polynomial-time solvable; NP means polynomial-time verifiable for YES answers.
- Reductions transfer tractability and hardness.
- NP-complete problems are the central hard problems inside NP .
- Approximation is a rigorous way to cope with hard optimization problems.

Next Time

Coming next

We step from complexity theory into modern machine learning optimization: gradient descent and training neural networks.

Algorithms for Data Science

Lecture #10: Gradient Descent and Neural Network Training

Mateusz Buczynski

University of Warsaw

03.06.2026

Thanks to dr Krzysztof Fleszar for the base material.

Machine Learning as Optimization

Core idea

Training a model means choosing parameters that make prediction error small.

Algorithmic translation

We must minimize a function of many variables.

Ingredients of Supervised Learning

Data

Training examples

$$(x_1, y_1), \dots, (x_n, y_n).$$

Model

Produces prediction $\hat{y} = f_{\theta}(x)$.

Parameters

θ are the weights and biases we want to learn.

Loss Function

Purpose

Measure how bad the predictions are.

Examples

- squared error for regression
- cross-entropy for classification

Training objective

Minimize the loss over the training data.

Why Not Just Try All Parameters?

Problem

The parameter space is usually huge and continuous.

Example

A neural network can have millions or billions of parameters.

So

We need an iterative optimization algorithm, not brute force.

Gradient: Local Slope Information

For one variable

The derivative tells us the local slope of the loss.

For many variables

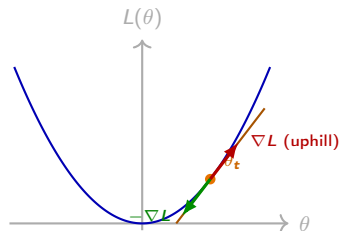
The gradient

$$\nabla L(\theta)$$

collects all partial derivatives.

Meaning

It points in the direction of steepest increase.



Why We Move in the Negative Gradient Direction

If the gradient points uphill

Then

$$-\nabla L(\theta)$$

points downhill.

Intuition

Take a small step in the direction that decreases the loss fastest locally.

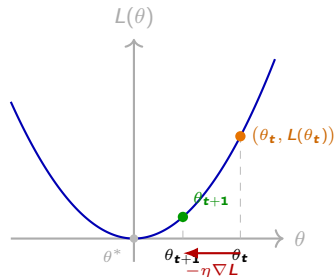
Gradient Descent Update Rule

Basic algorithm

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Parameters

- θ_t : current parameters
- η : learning rate (step size)
- $\nabla L(\theta_t)$: current gradient



Worked Example: Two Gradient Steps

Setup

$$L(w) = (w - 3)^2, \quad \nabla L(w) = 2(w - 3), \quad \eta = 0.5, \quad w_0 = 0$$

Step 1

$$\begin{aligned}\nabla L(w_0) &= 2(0 - 3) = -6 \\ w_1 &= 0 - 0.5 \cdot (-6) = 3\end{aligned}$$

Step 2

$$\begin{aligned}\nabla L(w_1) &= 2(3 - 3) = 0 \\ w_2 &= 3 - 0.5 \cdot 0 = 3\end{aligned}$$

Learning Rate

Too small

Very slow progress.

Too large

Overshooting, oscillation, or divergence.

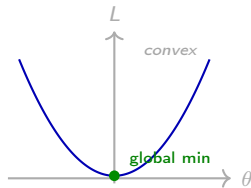
Engineering reality

Learning rate is one of the most important hyperparameters.

Convex vs Non-Convex Optimization

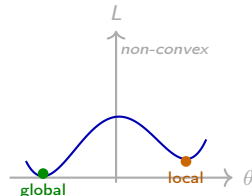
Convex loss

No bad local minima; gradient descent finds the global optimum.



Neural networks

Usually non-convex; may converge to a local minimum.



Example: Linear Regression

Model

$$\hat{y} = wx + b$$

Loss

Typically mean squared error.

Training

Gradient descent adjusts w and b to fit the data.

One Neuron as a Building Block

Computation

$$z = w^T x + b, \quad a = \sigma(z)$$

Interpretation

Weights combine inputs, bias shifts them, activation introduces nonlinearity.

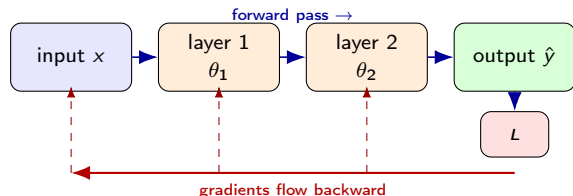
Why Neural Networks Need Backpropagation

Problem

The loss depends on parameters through many layers of intermediate computations.

Need

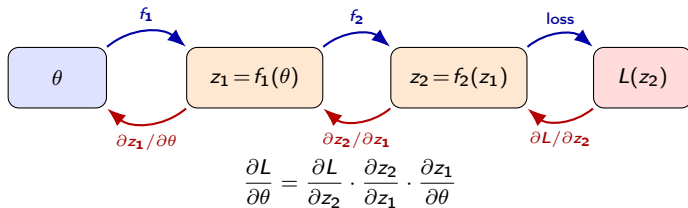
We must compute gradients of the final loss with respect to every parameter efficiently.



Chain Rule: The Core Mathematical Tool

Main principle

If a quantity depends on another quantity, derivatives multiply through the computational chain.



In neural networks

Backpropagation is repeated chain-rule application from the output back to earlier layers.

Backpropagation Intuition

Forward pass

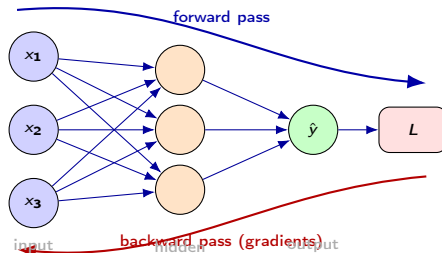
Compute predictions and the loss.

Backward pass

Propagate gradient information backward to compute parameter updates.

Why it matters

Without backpropagation, training deep networks would be far too expensive.



Backprop Walkthrough: Network Setup

Minimal network: one hidden unit



Formulas

$$z_1 = w_1 x \quad h = \sigma(z_1)$$

$$\hat{y} = w_2 h \quad L = \frac{1}{2}(\hat{y} - y)^2$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Concrete values

$$x = 1, \quad y = 0, \quad \eta = 0.5$$

$$w_1 = 1, \quad w_2 = 2$$

Backprop Step 1 — Forward Pass

Compute and store every intermediate value

$$z_1 = w_1 \cdot x = 1 \cdot 1 = 1$$

$$h = \sigma(z_1) = \sigma(1) \approx 0.73$$

$$\hat{y} = w_2 \cdot h = 2 \cdot 0.73 = 1.46$$

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(1.46)^2 \approx 1.07$$

Sigmoid at $z_1 = 1$

$$\sigma'(1) = 0.73 \cdot (1 - 0.73) \approx 0.20$$

Why store intermediates?

The backward pass reuses h , $\hat{y}$, and $\sigma'(z_1)$ — don't recompute.

Backprop Step 2 — Output Layer Gradient

Gradient of L with respect to w_2

$$\frac{\partial L}{\partial w_2} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\hat{y} - y} \cdot \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_h$$

Numbers

$$\frac{\partial L}{\partial w_2} = (1.46 - 0) \cdot 0.73 \approx 1.07$$

Save the upstream error signal

Define $\delta = \frac{\partial L}{\partial \hat{y}} = 1.46$. This scalar is passed backward into the hidden layer.

Backprop Step 3 — Hidden Layer Gradient

Chain rule extends back through h and z_1

$$\frac{\partial L}{\partial w_1} = \underbrace{\delta}_{1.46} \cdot \underbrace{\frac{\partial \hat{y}}{\partial h}}_{w_2=2} \cdot \underbrace{\frac{\partial h}{\partial z_1}}_{\sigma'(1) \approx 0.20} \cdot \underbrace{\frac{\partial z_1}{\partial w_1}}_{x=1}$$

Numbers

$$\frac{\partial L}{\partial w_1} = 1.46 \cdot 2 \cdot 0.20 \cdot 1 \approx 0.58$$

Pattern for deeper networks

Each extra layer adds one more local derivative to the product. The same δ is reused — this is why one backward pass suffices.

Backprop Step 4 — Weight Update

Apply gradient descent ($\eta = 0.5$)

$$w'_2 = w_2 - \eta \frac{\partial L}{\partial w_2} = 2 - 0.5 \cdot 1.07 = 1.47$$

$$w'_1 = w_1 - \eta \frac{\partial L}{\partial w_1} = 1 - 0.5 \cdot 0.58 = 0.71$$

Sanity check

Both weights decreased, so the network will predict a smaller $\hat{y}$, moving closer to the target $y = 0$.

One training iteration = four steps

Forward pass $\rightarrow$ loss $\rightarrow$ backward pass $\rightarrow$ update. Then repeat.

Training Loop for a Neural Network

- 1 initialize parameters
- 2 do a forward pass
- 3 compute the loss
- 4 do backpropagation
- 5 update parameters with gradient descent
- 6 repeat many times

Batch Gradient Descent

Rule

Use the full dataset to compute one exact gradient step.

Pros and cons

Stable but expensive for large datasets.

Stochastic Gradient Descent (SGD)

Rule

Use one randomly chosen example for each update.

Effect

Cheaper updates, but noisy gradient estimates.

Practical benefit

Can start improving quickly even on massive datasets.

Mini-Batch Gradient Descent

Compromise

Use a small batch of examples for each update.

Why this dominates practice

It balances hardware efficiency, gradient quality, and memory usage.

Common Training Problems

- loss decreases too slowly
- training becomes unstable
- gradients explode or vanish
- model overfits the training set

Message

Getting gradient descent to work well is both mathematics and engineering.

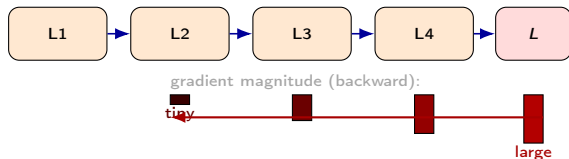
Vanishing and Exploding Gradients

What can go wrong

As gradients are propagated through many layers, they can shrink toward zero or grow uncontrollably.

Consequence

Early layers may learn too slowly or training may become numerically unstable.



Regularization and Generalization

Training loss is not the whole story

A model can fit the training data well but still perform poorly on unseen data.

Typical tools

Weight decay, dropout, early stopping, data augmentation.

When Do We Stop Training?

Typical criteria

- fixed number of epochs
- validation loss stops improving
- gradients or updates become very small

Best practice

Watch validation performance, not only training loss.

What to Remember

- Gradient descent is the standard template for training differentiable models.
- The gradient gives local information for improving the parameters.
- Backpropagation computes gradients efficiently in layered models.
- Mini-batch SGD is the practical default for large-scale neural training.
- Good training requires both optimization insight and engineering care.